

Лабораторна 6

Оцінювання руху: 2D–2D та 3D–3D відповідності

Покроковий гайд з виконання: теорія, математика, код, графіки

Що всередині:

- повне математичне підґрунтя: епіполярна геометрія, essential-матриця, її розклад, RANSAC;
- Deliverable 1 — алгоритм Ністера (5 точок): кроки, виродження, кількість ітерацій;
- Deliverable 2 — мінімальний 2-точковий розв’язувач з відомим обертанням;
- Deliverable 3–5 — готовий код для `pose_estimation.cpp` (OpenGV), метрики RPE, графіки;
- типові пастки з системами координат та як їх налагодити.

Зміст

1	Огляд лабораторної	4
1.1	Позначення	4
2	Математичне підґрунтя	5
2.1	Від пікселя до bearing vector	5
2.2	Еліполярне обмеження та essential-матриця	6
2.3	Розклад E на R і t : чотири розв'язки	7
2.4	RANSAC	8
3	Deliverable 1 — алгоритм Ністера (5 точок)	10
3.1	(a) Основні кроки	10
3.2	(b) Виродження	11
3.3	(c) Ітерації RANSAC	11
4	Deliverable 2 — мінімальний розв'язувач з відомим обертанням	13
4.1	Підрахунок ступенів свободи	13
4.2	Виведення	13
4.3	Псевдокод RANSAC (бонус)	14
5	Підготовка середовища	15
5.1	Збирання і дані	15
5.2	Як влаштований вузол	15
5.3	Змінні стенсилю pose_estimation.cpp	16
6	Deliverable 3 — калібрування ключових точок	19
7	Deliverable 4 — 2D–2D відповідності	20
7.1	Спільний шаблон RANSAC в OpenGV	20
7.2	5-point (Ністер) з RANSAC — case 0	21
7.3	5-point без RANSAC (бонус +5)	22
7.4	8-point (Longuet-Higgins) — case 1	22
7.5	2-point (відоме обертання) — case 2	23
7.6	Синхронізація ground truth з кадрами	24
7.7	Перший кадр	24
7.8	evaluateRPE: відносна похибка пози	25
7.9	Публікація оцінки	27
8	Deliverable 5 — 3D–3D відповідності (алгоритм Аруна)	28
8.1	Від пікселя до 3D-точки	28
8.2	Математика алгоритму Аруна	28
8.3	Код — case 3	29
8.4	Невалідна глибина	29
9	Запуск, графіки, перевірка результатів	31
9.1	Запуск усіх методів	31
9.2	Скрипт для графіків	31
9.3	Чого очікувати	32
9.4	Фактичні результати на vnav-lab6-office	33
9.5	Налагодження: типові симптоми	35
9.6	Чекліст задачі	35
A	Словник позначень	36
A.1	Системи координат і пози	36
A.2	Камера та вимірювання	36
A.3	Еліполярна геометрія	36
A.4	RANSAC	37
A.5	Алгоритм Ністера	37
A.6	2-точковий розв'язувач	37
A.7	Алгоритм Аруна	37

A.8 Метрики	38
-------------------	----

1 Огляд лабораторної

Мета — оцінити рух дрона між двома послідовними кадрами камери. Маємо два типи вхідних даних:

- **2D–2D**: тільки RGB-зображення. З відповідностей ключових точок (SIFT із Lab 5) відновлюємо відносне обертання R і **напрям** трансляції t (масштаб з однієї камери відновити неможливо).
- **3D–3D**: додатково маємо глибину кожного пікселя, тож кожна ключова точка стає 3D-точкою; відносна поза відновлюється повністю, разом з масштабом.

№	Завдання	Тип	Бали
1	Питання про алгоритм Ністера (5-point)	індивідуальне	20
2	Мінімальний розв’язувач з відомим обертанням (+ опціонально RANSAC, +5)	індивідуальне	15
3	Калібрування ключових точок (calibrateKeypoints)	командне	5
4	5-point, 8-point, 2-point з RANSAC + RPE + графіки	командне	45
5	Алгоритм Аруна (3D–3D) з RANSAC + графіки	командне	20

Файли, з якими працюємо (у цьому репозиторії):

VNAV-labs/lab6/src/pose_estimation.cpp	ROS 2 вузол; сюди пишемо весь код Deliverables 3–5
VNAV-labs/lab6/include/lab6_utils.h	cv2Pose, eigen2Pose, extractPose, tf2TransformToIsometry
VNAV-labs/lab6/launch/video_tracking.launch.yaml	запуск вузла + RViz2 + програвання rosbag
ros2-docker/download_lab6_bags.sh	завантаження і конвертація vnav-lab6-office у rosbag2

1.1 Позначення

Щоб не заплутатися (а в цій лабі **вся** складність — у системах координат), домовимося:

- $T_B^A \in \text{SE}(3)$ — поза системи B , виражена в системі A . Точка з координатами p_B у системі B має в системі A координати $p_A = R_B^A p_B + t_B^A$. Композиція: $T_C^A = T_B^A T_C^B$, обернення: $(T_B^A)^{-1} = T_A^B$.
- Кадр 1 — **попередній** (час $t-1$), кадр 2 — **поточний** (час t). Саме так нумерує OpenGV: bearing_vector_1 — попередній кадр, bearing_vector_2 — поточний.
- $R = R_2^1$, $t = t_2^1$ — обертання й положення центру камери 2, виражені в камері 1. OpenGV повертає саме цю пару у вигляді матриці $3 \times 4 [R \mid t]$.
- $f \in \mathbb{R}^3$, $\|f\| = 1$ — **bearing vector** (одичинний вектор напрямку на точку з центру камери).
- $[a]_\times$ — кососиметрична матриця векторного добутку: $[a]_\times b = a \times b$,

$$[a]_\times = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}. \quad (1)$$

- Жирні малі літери (p, t, f) — вектори-стовпці в $\mathbb{R}^3$; великі (R, E, K, A) — матриці; звичайні малі (x, λ, s) — скаляри. $a^\top$ — транспонування, $\|a\|$ — евклідова довжина, $a \cdot b$ — скалярний добуток, $a \times b$ — векторний добуток.
- Після кожної ключової формули є блок «де:» з поясненням усіх її змінних, змінні коду пояснено в розділі «Змінні стенсилю» (5.3) і під кожним фрагментом коду, а в кінці документа є зведений словник усіх математичних позначень (додаток А).

2 Математичне підґрунтя

2.1 Від пікселя до bearing vector

Модель камери-обскури: 3D-точка $\mathbf{p} = (X, Y, Z)^\top$ у системі камери проєктується в піксель

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K\mathbf{p}, \quad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad \lambda = Z. \quad (2)$$

У нашому симуляторі $f_x = f_y = 415.692$, $c_x = 360$, $c_y = 240$ (зображення 720×480), дисторсія нульова.

де:

$\mathbf{p} = (X, Y, Z)^\top$	координати 3D-точки в системі камери, метри: X — вправо, Y — вниз, Z — вздовж оптичної осі (вперед, «глибина»)
(u, v)	координати пікселя: u — стовпець (зліва направо), v — рядок (згори вниз)
λ	проєктивний множник; для камери-обскури дорівнює глибині Z (однорідні координати визначені з точністю до множника)
K	матриця внутрішніх параметрів (intrinsics), у коді <code>camera_params_.K</code>
f_x, f_y	фокусні відстані в пікселях по горизонталі й вертикалі (скільки пікселів відповідає одиниці на площині $Z = 1$)
c_x, c_y	головна точка (principal point) — піксель, куди проєктується оптична вісь; зазвичай центр зображення

Обертаючи рівняння, отримуємо **нормалізовані координати**

$$\mathbf{x} = K^{-1} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{pmatrix} = \frac{\mathbf{p}}{Z}. \quad (3)$$

Вони лежать на площині $Z = 1$ («камера з фокусною відстанню 1»). Bearing vector — це той самий напрямок, нормований на одиницю:

$$\mathbf{f} = \frac{\mathbf{x}}{\|\mathbf{x}\|} = \frac{\mathbf{p}}{\|\mathbf{p}\|}. \quad (4)$$

де:

$\mathbf{x} = (x, y, 1)^\top$	нормалізовані (калібровані) координати — точка на віртуальній площині $Z = 1$; не залежать від K
$\mathbf{f}$	bearing vector — одиничний вектор з центру камери на точку; відомий лише напрям , не відстань
$\ \cdot\ $	евклідова норма (довжина вектора)

OpenGV працює саме з bearing vectors, а не з пікселями: так формули не залежать від конкретної камери, і навіть ширококутні/сферичні камери обробляються однаково.

2.2 Епіполярне обмеження та essential-матриця

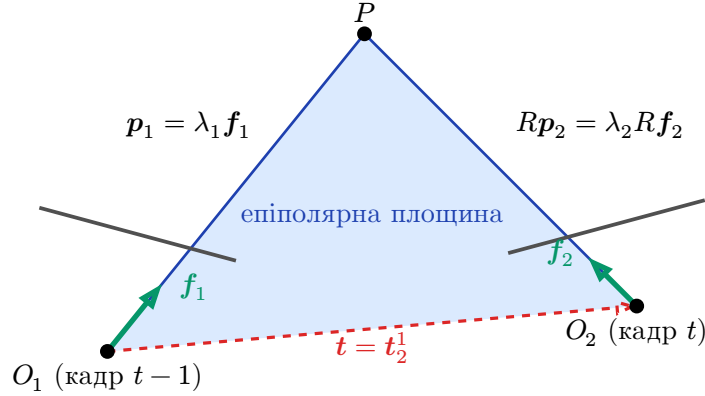


Рисунок 1: Епіполярна геометрія: O_1 , O_2 і P задають площину, в якій лежать обидва промені.

Нехай точка P має координати p_1 у камері 1 та p_2 у камері 2. Тоді

$$p_1 = Rp_2 + t. \quad (5)$$

де:

P	фізична 3D-точка сцени (одна й та сама для обох кадрів)
O_1, O_2	оптичні центри камери в момент $t-1$ (кадр 1) та t (кадр 2)
p_1, p_2	координати P у системі камери 1 і камери 2 відповідно (метри)
$R = R_2^1$	матриця обертання 3×3 ($R^T R = I$, $\det R = 1$), що переводить напрямки з системи 2 в систему 1; її стовпці — осі камери 2, виражені в осях камери 1
$t = t_2^1$	положення центру O_2 у системі камери 1 — базова лінія (baseline); у 2D-2D відоме лише з точністю до масштабу

Три вектори, виражені в системі камери 1 і відкладені з O_1 — p_1 (на точку), t (на O_2) та $Rp_2 = p_1 - t$ (від O_2 на точку) — лежать в одній площині (Рисунок 1). Умова компланарності — нульовий мішаний добуток:

$$p_1 \cdot (t \times Rp_2) = 0 \iff p_1^T [t]_{\times} Rp_2 = 0. \quad (6)$$

Ділимо на $\|p_1\| \|p_2\| > 0$ і отримуємо **епіполярне обмеження** для bearing vectors:

$$f_1^T E f_2 = 0, \quad E = [t]_{\times} R. \quad (7)$$

де:

f_1, f_2	bearing vectors однієї й тієї ж точки P у кадрі 1 і кадрі 2 (одна відповідність , correspondence)
E	essential-матриця 3×3 — кодує відносну позу (R, t) з точністю до масштабу
$[t]_{\times}$	косиметрична матриця, що реалізує векторний добуток з t
λ_1, λ_2	(на рисунку) глибини вздовж променів: $p_1 = \lambda_1 f_1$, $p_2 = \lambda_2 f_2$; $\lambda_k = \ p_k\ > 0$

Примітка

OpenGV використовує саме конвенцію 7 ($f_1^T E f_2 = 0$). OpenCV (`findEssentialMat`, `recoverPose`) — навпаки: $x_2^T E x_1 = 0$. Про наслідки — у розділі про 5-point без RANSAC.

2.2.1 Властивості E

1. Масштаб. Рівняння 7 однорідне: E і αE задають ті самі обмеження. Отже t визначається лише з точністю до масштабу. Ступенів свободи: 3 (обертання) + 3 (трансляція) — 1 (масштаб) = **5**. Кожна відповідність дає **одне** скалярне рівняння, тому мінімально потрібно 5 точок.

2. Сингулярні числа $(\sigma, \sigma, 0)$. Для одиничного t матриця $[t]_{\times}$ має сингулярні числа $(1, 1, 0)$ (вона повертає ортогональне до t доповнення на 90° і анулює t). Множення справа на обертання сингулярних чисел не змінює. Звідси $\det E = 0$ та два рівних ненульових сингулярних числа.

3. Кубічні обмеження (trace constraint). Для будь-якої essential-матриці

$$2EE^\top E - \text{tr}(EE^\top)E = 0. \quad (8)$$

Доведення. $EE^\top = [t]_\times R R^\top [t]_\times^\top = -[t]_\times^2 = \|t\|^2 I - tt^\top$ (відома тотожність для косиметричної матриці). Тоді

$$EE^\top E = (\|t\|^2 I - tt^\top)[t]_\times R = \|t\|^2 E - \underbrace{t t^\top [t]_\times}_{=0^\top} R = \|t\|^2 E, \quad (9)$$

а $\text{tr}(EE^\top) = 3\|t\|^2 - \|t\|^2 = 2\|t\|^2$. Підставляємо: $2\|t\|^2 E - 2\|t\|^2 E = 0$. $\square$

Рівняння 8 — це 9 скалярних кубічних рівнянь; разом з $\det E = 0$ вони характеризують множину essential-матриць. Їх використовує алгоритм Ністера.

де:

σ — ненульове сингулярне число E (дорівнює $\|t\|$); E має сингулярні числа $(\sigma, \sigma, 0)$
 $\text{tr}(\cdot)$ — слід матриці — сума діагональних елементів
 I — одинична матриця 3×3
 $tt^\top$ — зовнішній добуток — матриця 3×3 рангу 1 (проектор на напрям t , помножений на $\|t\|^2$)

2.2.2 Лінійна форма епіполярного обмеження

Розпишемо 7 поелементно: $\sum_{i,j} f_{1,i} E_{ij} f_{2,j} = 0$. Якщо записати E по рядках у вектор $e = (E_{11}, E_{12}, E_{13}, E_{21}, \dots, E_{33})^\top \in \mathbb{R}^9$, то

$$f_1^\top E f_2 = (f_1 \otimes f_2)^\top e = a^\top e, \quad a = (f_{1,1} f_{2,1}, f_{1,1} f_{2,2}, \dots, f_{1,3} f_{2,3})^\top. \quad (10)$$

Для N відповідностей складаємо матрицю $A \in \mathbb{R}^{N \times 9}$ з рядками $a_i^\top$, і задача стає **лінійною**: $Ae = 0$. На цьому побудовані і 8-точковий, і 5-точковий алгоритми.

де:

$e \in \mathbb{R}^9$ — невідомі елементи E , записані в один стовпець рядок за рядком
 $f_{1,i}, f_{2,j}$ — i -та компонента f_1 та j -та компонента f_2 ($i, j \in \{1, 2, 3\}$)
 E_{ij} — елемент E у рядку i , стовпці j
 $\otimes$ — добуток Кронекера: $f_1 \otimes f_2$ — вектор з 9 усіх попарних добутків $f_{1,i} f_{2,j}$
 a_i — вектор коефіцієнтів епіполярного рівняння i -ї відповідності
 A — матриця системи, рядки — $a_i^\top$; кожна відповідність — один рядок
 N — кількість відповідностей (у коді N — кількість SIFT-матчів)

2.3 Розклад E на R і t : чотири розв'язки

Нехай $E = U \text{diag}(1, 1, 0) V^\top$ — SVD, де знаки U, V вибрано так, щоб $\det U = \det V = +1$. Введемо

$$W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (\text{обертання на } 90^\circ \text{ навколо } z). \quad (11)$$

Тоді всі розклади $E = [t]_\times R$ мають вигляд

$$R \in \{U W V^\top, U W^\top V^\top\}, \quad t \in \{+u_3, -u_3\}, \quad (12)$$

де u_3 — третій стовпець U (ліве ядро E : $u_3^\top E = 0$, а для $E = [t]_\times R$ ліве ядро — саме t).

Отримуємо **4 кандидати**. Геометрично: зміна знаку t — дзеркальне розташування камер; заміна $W \rightarrow W^\top$ — «twisted pair», друга камера повернута на 180° навколо базової лінії. Лише в одному з чотирьох варіантів точки лежать **перед** обома камерами.

де:

U, V — ортогональні матриці 3×3 з SVD: $E = U \Sigma V^\top$; стовпці U — ліві, стовпці V — праві сингулярні вектори

$\text{diag}(a, b, c)$	діагональна матриця з a, b, c на діагоналі (тут — сингулярні числа E , нормовані)
W	фіксована матриця обертання на 90° навколо осі z ; допоміжна конструкція для розкладу
u_3	третій стовпець U — одиничний вектор, що задає напрям t (з точністю до знаку)

Перевірка хіральності (cheirality). Для відповідності шукаємо глибини λ_1, λ_2 з $\lambda_1 f_1 = \lambda_2 R f_2 + t$, тобто методом найменших квадратів

$$(f_1 \ -Rf_2) \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix} = t. \quad (13)$$

Правильний кандидат — той, для якого $\lambda_1 > 0$ і $\lambda_2 > 0$ для більшості точок. Саме так чинить `CentralRelativePoseSacProblem` в `OpenGV` (і `cv::recoverPose` в `OpenCV`).

де:

λ_1, λ_2	невідомі глибини точки вздовж променів f_1 (у камері 1) та f_2 (у камері 2); від’ємна глибина означає, що точка «позаду» камери — фізично неможливо
$(f_1 \ -Rf_2)$	матриця 3×2 зі стовпцями f_1 та $-Rf_2$; система переозначена (3 рівняння, 2 невідомі), тому розв’язується методом найменших квадратів

2.4 RANSAC

SIFT-відповідності містять хибні пари (outliers). Мінімізація за всіма точками (least squares) ламається навіть від одного грубого викиду. RANSAC:

1. Випадково вибрати мінімальну вибірку з s відповідностей.
2. Розв’язати мінімальну задачу $\rightarrow$ одна чи кілька гіпотез моделі.
3. Для кожної гіпотези порахувати похибку всіх N точок; точки з похибкою $< \tau$ — inliers.
4. Запам’ятати гіпотезу з найбільшою кількістю inliers.
5. Повторювати, поки не вичерпано ліміт ітерацій (або адаптивний критерій зупинки).
6. (Опціонально) переоцінити модель за всіма inliers.

2.4.1 Кількість ітерацій

Нехай частка outliers — ε , частка inliers — $w = 1 - \varepsilon$. Імовірність, що вибірка з s точок (з поверненням, наближено) складається лише з inliers: w^s .

Очікувана кількість ітерацій до першої «чистої» вибірки. Номер першого успіху k має геометричний розподіл з імовірністю успіху $q = w^s$:

$$\mathbb{P}(k) = (1 - q)^{k-1} q, \quad \mathbb{E}[k] = \sum_{k=1}^{\infty} k(1 - q)^{k-1} q = \frac{1}{q} = \frac{1}{w^s}. \quad (14)$$

Кількість ітерацій для заданої надійності p . Імовірність, що всі N_{it} вибірок «брудні»: $(1 - w^s)^{N_{\text{it}}}$. Вимагаємо $(1 - w^s)^{N_{\text{it}}} \leq 1 - p$ і логарифмуємо (обидва логарифми від’ємні, тому знак нерівності перевертається):

$$N_{\text{it}} \geq \frac{\log(1 - p)}{\log(1 - w^s)}. \quad (15)$$

де:

ε	частка outliers серед усіх відповідностей (хибних SIFT-матчів), $0 \leq \varepsilon < 1$
$w = 1 - \varepsilon$	частка inliers — імовірність, що випадково взята відповідність правильна
s	розмір мінімальної вибірки (скільки точок потрібно розв’язувачу): 5 — Ністер, 8 — Longuet-Higgins, 2 — відоме R , 3 — Арун
$q = w^s$	імовірність, що вся вибірка з s точок чиста (без жодного outlier)
k	номер ітерації, на якій уперше трапилась чиста вибірка (випадкова величина)
$\mathbb{P}(k), \mathbb{E}[k]$	імовірність того, що перший успіх саме на ітерації k , та математичне сподівання k
p	бажана надійність — імовірність того, що хоча б одна вибірка буде чистою (зазвичай 0.99); у коді <code>ransac.probability_</code>

N_{it}	кількість ітерацій RANSAC, достатня для надійності p ; у кодї верхня межа — <code>ransac.max_iterations_</code>
τ	поріг похибки, нижче якого точка вважається inlier; у кодї <code>ransac.threshold_</code>

Мінімальна ви- бірка s	$\mathbb{E}[k]$ при $\varepsilon = 0.5$	N_{it} при $\varepsilon = 0.5, p = 0.99$	$\mathbb{E}[k]$ при $\varepsilon = 0.3$	N_{it} при $\varepsilon = 0.3, p = 0.99$
2 (2-point)	4	17	2.0	7
3 (Арун)	8	35	2.9	11
5 (Ністер)	32	146	5.9	26
8 (Longuet-Higgins)	256	1177	17.3	78

Висновок, який варто запам'ятати: кількість ітерацій росте **експоненційно** по s . Саме тому мінімальні розв'язувачі такі цінні — кожна зекономлена точка в мінімальній вибірці зменшує N_{it} приблизно в $\frac{1}{w}$ разів.

2.4.2 Попір inliers в OpenGV

Для відносної пози OpenGV триангулює точку за гіпотезою (R, t) , перепроєктує її в обидві камери і рахує

$$\text{score} = (1 - \mathbf{f}_1^\top \hat{\mathbf{f}}_1) + (1 - \mathbf{f}_2^\top \hat{\mathbf{f}}_2) = (1 - \cos \alpha_1) + (1 - \cos \alpha_2), \quad (16)$$

де α_k — кут між виміряним і перепроєктованим bearing vector. Для малих кутів $1 - \cos \alpha \approx \alpha^2/2$, а похибка в δ пікселів відповідає $\alpha \approx \arctan(\delta/f)$. Тому поріг задаємо так:

$$\tau = 2(1 - \cos(\arctan(\delta/f))). \quad (17)$$

При $f = 415.7$: $\delta = 1 \text{ px} \rightarrow 1 - \cos \alpha \approx 2.9 \cdot 10^{-6}$; $\delta = 2 \text{ px} \rightarrow 1.16 \cdot 10^{-5}$. Для 3D-3D (PointCloudSacProblem) похибка — просто евклідова відстань $\|\mathbf{p}_1 - (R\mathbf{p}_2 + \mathbf{t})\|$ у метрах.

де:

$\hat{\mathbf{f}}_1, \hat{\mathbf{f}}_2$	перепроєктовані bearing vectors: напрямки на триангульовану за гіпотезою (R, t) точку; для ідеальної гіпотези і без шуму $\hat{\mathbf{f}}_k = \mathbf{f}_k$
α_1, α_2	кутова похибка (радіани) між виміряним і перепроєктованим напрямком у кадрі 1 і 2
score	сумарна похибка однієї відповідності, $\in [0, 4]$; порівнюється з τ
δ	допустима похибка локалізації ключової точки в пікселях (1–2 px для SIFT)
f	тут — фокусна відстань у пікселях ($f = f_x = 415.7$), не bearing vector

3 Deliverable 1 — алгоритм Ністера (5 точок)

Deliverable 1 (20 балів)

Прочитати D. Nistér, «*An efficient solution to the five-point relative pose problem*», CVPR 2003, та відповісти: (а) основні обчислювальні кроки; (б) чи має алгоритм виродження; (с) очікувана кількість ітерацій RANSAC.

3.1 (а) Основні кроки

Вхід — 5 відповістей у **каліброваних** координатах ($\mathbf{q}_i = K^{-1}(u, v, 1)^\top$ або bearing vectors). Вихід — до 10 essential-матриць і відповідні $(R, \mathbf{t})$.

Крок 1. Лінійна частина: 4-вимірне ядро. Кожна відповідність дає рядок $\mathbf{a}_i^\top = (\mathbf{f}_{1,i} \otimes \mathbf{f}_{2,i})^\top$. Матриця $A \in \mathbb{R}^{5 \times 9}$ має (у загальному положенні) ранг 5, отже її ядро 4-вимірне. Базис ядра $\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{w}$ отримуємо через SVD або QR (Ністер використовує QR — швидше). Кожна E , що задовольняє п'ять епіпольярних рівнянь, має вигляд

$$E = \mathbf{x}X + \mathbf{y}Y + \mathbf{z}Z + \mathbf{w}W, \quad (18)$$

де X, Y, Z, W — вектори базису, перетворені назад у матриці 3×3 . Оскільки E визначена з точністю до масштабу, кладемо $w = 1$. Залишилось **3 невідомі** x, y, z .

де:

$\mathbf{q}_i$	калібровані координати i -ї точки (те саме, що $\mathbf{x}$ з розділу 2.1; bearing vector з точністю до довжини)
$\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{w} \in \mathbb{R}^9$	базис 4-вимірного ядра матриці A (4 останні праві сингулярні вектори)
X, Y, Z, W	ті самі вектори базису, розкладені в матриці 3×3 . Увага: цей W не має нічого спільного з матрицею W з розкладу E у розділі 2.3 — так позначено в статті Ністера
x, y, z, w	невідомі скалярні коефіцієнти лінійної комбінації (не плутати з координатами точки); $w = 1$ фіксує масштаб

Крок 2. Нелінійні обмеження → 10 кубічних рівнянь. Підставляємо параметризацію в

- $\det E = 0$ — 1 рівняння,
- $2EE^\top E - \text{tr}(EE^\top)E = 0$ — 9 рівнянь.

Кожен елемент E лінійний по (x, y, z) , тож усі 10 рівнянь **кубічні**. Мономів степеня ≤ 3 від трьох змінних — $\binom{3+3}{3} = 20$:

$$x^3, y^3, x^2y, xy^2, x^2z, x^2, y^2z, y^2, xyz, xy, xz^2, xz, x, yz^2, yz, y, z^3, z^2, z, 1. \quad (19)$$

Записуємо систему як $M\mathbf{m} = 0$, де $M \in \mathbb{R}^{10 \times 20}$, $\mathbf{m}$ — вектор мономів.

де:

M	матриця коефіцієнтів: рядок — одне з 10 кубічних рівнянь, стовпець — один з 20 мономів; числа залежать лише від X, Y, Z, W
$\mathbf{m}$	вектор 20 мономів $(x^3, y^3, x^2y, \dots, z, 1)^\top$ — невідомий, але з відомою структурою

Крок 3. Виключення Гауса–Жордана. Зводимо M до вигляду $[I_{10} \mid B]$ (мономи впорядковані так, що перші 10 містять x і y у старших степенях). Після цього кожне рівняння виражає «старший» моном через молодші. Віднімаючи рядки, домножені на z (наприклад, $\langle k \rangle - z\langle l \rangle$), позбавляємося мономів з x^2, xy, y^2 і отримуємо **3 рівняння**, лінійних по $x, y, 1$ з коефіцієнтами-поліномами від z :

$$B(z) \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \mathbf{0}, \quad B(z) \in \mathbb{R}[z]^{3 \times 3}, \quad (20)$$

де елементи B — поліноми степенів 3, 3 і 4 по z .

де:

I_{10}	одинична матриця 10×10 (результат зведення лівої частини M)
$\langle k \rangle$	позначення k -го рядка (рівняння) зведеної матриці, як у статті Ністера
$B(z)$	матриця 3×3 , кожен елемент якої — поліном від z ; при фіксованому z — звичайна числова матриця
$\mathbb{R}[z]$	множина поліномів від z з дійсними коефіцієнтами

Крок 4. Поліном 10-го степеня. Нетривіальний розв'язок $(x, y, 1)$ існує лише якщо

$$\det B(z) = 0, \quad (21)$$

а це поліном **10-го степеня** по z . Його дійсні корені знаходять через послідовності Штурма + бісекцію (Ністер) або через власні числа супровідної матриці. Дійсних коренів ≤ 10 (класичний результат: задача 5 точок має до 10 розв'язків).

Крок 5. Відновлення E . Для кожного кореня z_k : $(x_k, y_k, 1)$ — ядро $B(z_k)$ (векторний добуток двох рядків), далі $E_k = x_k X + y_k Y + z_k Z + W$. Тут k — номер дійсного кореня ($k \leq 10$), а x_k, y_k, z_k — відповідні значення коефіцієнтів.

Крок 6. $E \rightarrow (R, t)$. Розклад через SVD (розділ 2.3) дає 4 кандидати на кожну E_k ; перевірка хіральності лишає один. Отримуємо до 10 гіпотез (R, t) .

Крок 7. Вибір серед гіпотез. Кілька дійсних розв'язків розрізнити за 5 точками неможливо — потрібні додаткові точки. У RANSAC це відбувається природно: кожна гіпотеза оцінюється кількістю inliers серед усіх N точок.

3.2 (b) Виродження

1. **Чисте обертання / нульова база ($t = 0$).** Тоді $E = [0]_{\times} R = 0$: епіполярне обмеження виконується тотожно, t не визначене, а R з E не відновлюється. На практиці небезпечніша **майже** нульова база (дрон зависає або кадри йдуть дуже часто): напрям t стає погано обумовленим, шум у пікселях сильно «крутить» t . Для нашого rosbag це актуально — частота кадрів висока.
2. **Критичні поверхні.** Якщо 3D-точки і **обидва** центри камер лежать на одній лінійчатій квадриці (наприклад, однопорожнинному гіперболоїді), існує кілька фізично коректних розв'язків — неоднозначність не усувається жодною кількістю точок з цієї поверхні.
3. **Плоска сцена — важлива відмінність від 8-point.** Ністер показує, що 5-точковий алгоритм **не** ламається, коли всі точки лежать на площині (на відміну від 7- і 8-точкових методів, у яких ранг A падає і ядро стає більш ніж одновимірним). Проте для плоскої сцени загалом існують **два** геометрично допустимі розв'язки (класична двозначність гомографії), які не розрізнити додатковими точками з тієї ж площини — потрібні неколінеарні з площиною точки або третій вид.
4. **Вироджені конфігурації самих 5 точок.** Колінеарні точки, точки, що збігаються, або точка в напрямку базової лінії (епіполь: $f_1 \parallel t \Rightarrow f_1^T[t]_{\times} = 0$ і рівняння тривіальне). Тоді ранг $A < 5$, ядро має вимірність > 4 .
5. **Майже-виродження (bas-relief).** При вузькому полі зору і бічному русі обертання й трансляцію важко розрізнити: малий поворот майже еквівалентний малому зсуву. Алгоритм формально працює, але оцінка чутлива до шуму.

3.3 (c) Ітерації RANSAC

Мінімальна вибірка $s = 5$. У припущеннях лекцій (частка outliers $\varepsilon = 0.5$, бажана надійність $p = 0.99$):

- **очікувана** кількість вибірок до першої без outliers: $\mathbb{E}[k] = 1/(1 - \varepsilon)^5 = 2^5 = \mathbf{32}$;
- кількість ітерацій, щоб знайти чисту вибірку з імовірністю 0.99 (формула 15):

$$N_{it} = \left\lceil \frac{\log(1 - 0.99)}{\log(1 - 0.5^5)} \right\rceil = \left\lceil \frac{-4.605}{-0.03175} \right\rceil = \mathbf{146}. \quad (22)$$

Для порівняння, 8-точковий алгоритм у тих самих умовах потребує 1177 ітерацій — у 8 разів більше. Зауважимо, що кожна ітерація Ністера дає до 10 гіпотез, які треба оцінити, але це все одно значно дешевше.

Порада

У відповіді варто явно вписати обидві величини ($\mathbb{E}[k] = 32$ та $N_{\text{it}} = 146$) і загальні формули — тоді відповідь коректна за будь-якого трактування «expected number of iterations».

4 Deliverable 2 — мінімальний розв’язувач з відомим обертанням

Deliverable 2 (15 балів)

Обертання R між t_1 і t_2 відоме з гіроскопа IMU. Побудувати мінімальний розв’язувач для решти ступенів свободи (поза з точністю до масштабу). Опціонально (+5): псевдокод RANSAC з цим розв’язувачем.

4.1 Підрахунок ступенів свободи

Невідома лише $\mathbf{t} \in \mathbb{R}^3$, і лише з точністю до масштабу, тобто $\mathbf{t}/\|\mathbf{t}\| \in S^2$ — **2 ступені свободи**. Кожна відповідність дає одне скалярне рівняння, тому мінімальний розв’язувач потребує **2 точки**.

4.2 Виведення

Позначимо «розвернутий» bearing vector другого кадру $\mathbf{g}_i = R\mathbf{f}_{2,i}$ (обидва вектори тепер у системі камери 1). Епіполярне обмеження 7:

$$\mathbf{f}_{1,i}^\top [\mathbf{t}]_\times \mathbf{g}_i = \mathbf{f}_{1,i} \cdot (\mathbf{t} \times \mathbf{g}_i) = 0. \quad (23)$$

Циклічна перестановка мішаного добутку, $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathbf{b} \cdot (\mathbf{c} \times \mathbf{a})$, дає

$$\mathbf{t} \cdot (\mathbf{g}_i \times \mathbf{f}_{1,i}) = 0 \iff \mathbf{n}_i^\top \mathbf{t} = 0, \quad \mathbf{n}_i = \mathbf{f}_{1,i} \times R\mathbf{f}_{2,i}. \quad (24)$$

де:

i	номер відповідності ($i = 1, \dots, N$)
$\mathbf{f}_{1,i}, \mathbf{f}_{2,i}$	bearing vectors i -ї відповідності в кадри 1 і 2
R	відоме відносно обертання R_2^1 (з гіроскопа; у лабі — з ground truth)
$\mathbf{g}_i = R\mathbf{f}_{2,i}$	«розвернутий» промінь другого кадру — той самий напрям, але виражений в осях камери 1; після цього між кадрами лишається лише зсув
$\mathbf{n}_i$	нормаль до i -ї епіполярної площини (площини через O_1, O_2 і P_i); не одинична, її довжина = $\sin$ кута між $\mathbf{f}_{1,i}$ та $\mathbf{g}_i$
$\mathbf{t}$	невідомий напрям руху (базова лінія) у системі камери 1

Геометрично: $\mathbf{n}_i$ — нормаль до епіполярної площини i -ї точки. Базова лінія $\mathbf{t}$ лежить у **кожній** епіполярній площині, тобто ортогональна до всіх нормалей. Обмеження 24 — **лінійне** по $\mathbf{t}$ (на відміну від загального випадку, де R невідоме).

Дві точки дають дві нормалі; вектор, ортогональний до обох:

$$\hat{\mathbf{t}} = \pm \frac{\mathbf{n}_1 \times \mathbf{n}_2}{\|\mathbf{n}_1 \times \mathbf{n}_2\|}. \quad (25)$$

де:

$\hat{\mathbf{t}}$	оцінка напрямку трансляції, одиничний вектор ($\ \hat{\mathbf{t}}\ = 1$ — так фіксуємо невідомий масштаб)
$\mathbf{n}_1, \mathbf{n}_2$	нормалі епіполярних площин двох вибраних точок
$\pm$	знак не визначається епіполярним рівнянням; обирається перевіркою хіральності

Вибір знаку (хіральність). $\pm \hat{\mathbf{t}}$ обидва задовольняють 24. Для кожного знаку розв’язуємо $(\mathbf{f}_1 - \mathbf{g})(\lambda_1, \lambda_2)^\top = \hat{\mathbf{t}}$ (найменші квадрати). Зміна знаку $\hat{\mathbf{t}}$ змінює знак обох глибин, тож рівно один варіант дає $\lambda_1, \lambda_2 > 0$ — його й беремо.

Порівняння з Ністером: один розв’язок замість до 10, жодних поліномів — лише два векторні добутки. Вартість — $O(1)$, кількість ітерацій RANSAC при $\varepsilon = 0.5$: 17 замість 146.

Немінімальна версія (для уточнення за inliers): мінімізуємо $\sum_i (\mathbf{n}_i^\top \mathbf{t})^2 = \mathbf{t}^\top \left(\sum_i \mathbf{n}_i \mathbf{n}_i^\top \right) \mathbf{t}$ при $\|\mathbf{t}\| = 1$. Розв’язок — власний вектор матриці 3×3 $M = \sum_i \mathbf{n}_i \mathbf{n}_i^\top$ з найменшим власним числом (задача Релея). Тут M — симетрична невід’ємно визначена матриця «розкиду» нормалей (не

плутати з M з алгоритму Ністера); її найменше власне число — це залишкова сума квадратів $\sum_i (\mathbf{n}_i^\top \hat{\mathbf{t}})^2$, для ідеальних даних $= 0$.

Виродження:

- $\mathbf{n}_1 \parallel \mathbf{n}_2$: обидві 3D-точки лежать в одній епіполярній площині (тобто компланарні з базовою лінією) — тоді $\mathbf{n}_1 \times \mathbf{n}_2 = 0$;
- $\mathbf{n}_i = 0$: точка в напрямку епіполя ($\mathbf{f}_1 \parallel R\mathbf{f}_2$), зокрема дуже далекі точки при малій базі;
- $\mathbf{t} = 0$ (чисте обертання) — усі $\mathbf{n}_i \approx 0$, напрямок не визначений;
- похибка в R з IMU (дрейф гіроскопа) напряму переходить у похибку $\mathbf{t}$: розв’язувач «вірить» R беззастережно.

4.3 Псевдокод RANSAC (бонус)

```

Вхід: {(f1_i, f2_i)}_{i=1..N} – bearing vectors; R – обертання з IMU;
      tau – поріг; p – надійність (напр. 0.99)
Вихід: t_best (одичинний), множина inliers

g_i ← R · f2_i           для всіх i           // розвертаємо один раз
n_i ← f1_i × g_i         для всіх i           // нормалі епіполярних площин
best ← ∅; K ← ∞; k ← 0
while k < K:
    вибрати випадково i ≠ j
    c ← n_i × n_j
    if ||c|| < eps: k ← k+1; continue           // вироджена пара
    t ← c / ||c||
    if глибини λ1, λ2 для точки i не обидві > 0: t ← -t // хіральність
    I ← { m : err(t, m) < tau }                 // err: кут до епіп. площини,
                                                // напр. |n_m·t| / ||n_m||,
                                                // + перевірка λ>0 для точки m

    if |I| > |best|:
        best ← I; t_best ← t
        w ← |I| / N
        K ← log(1 - p) / log(1 - w^2)           // адаптивна зупинка, s = 2
    k ← k + 1
// уточнення: t_best ← власний вектор min λ матриці Σ_{m∈best} n_m n_m^T (+ знак)
return t_best, best

```

де:

tau, p	поріг inlier і бажана надійність
eps	мале число ($\approx 10^{-9}$) для відсіювання виродженої пари
K, k	поточна потрібна кількість ітерацій (адаптивна, спершу ∞) і лічильник виконаних
c	ненормований векторний добуток нормалей $\mathbf{n}_i \times \mathbf{n}_j$
I, best	множина inliers поточної гіпотези і найкраща знайдена множина
w	поточна оцінка частки inliers $ I / N$
m	індекс точки, що перевіряється

Похибка $|\mathbf{n}_m^\top \mathbf{t}| / \|\mathbf{n}_m\| = |\sin \beta|$, де β — кут між $\mathbf{t}$ та епіполярною площиною m -ї точки, — дешева геометрична міра. Можна також (як OpenGV) тріангулювати і міряти кут перепроєкції.

5 Підготовка середовища

5.1 Збирання і дані

У цьому репозиторії все вже налаштовано в Docker (OpenGV зібраний в образі, lab_5 збирається разом з lab_6):

```
# на хості, з кореня репозиторію
bash ros2-docker/download_lab6_bags.sh      # один раз: vnav-lab6-office → rosbag2
bash ros2-docker/run_lab_dev.sh lab6 --build-only
bash ros2-docker/run_lab_dev.sh lab6        # збирає lab_5 + lab_6 і запускає launch
```

Перезбирання всередині контейнера після змін у коді:

```
source /opt/ros/humble/setup.bash
cd /workspace
colcon build --symlink-install --packages-select lab_5 lab_6
source install/setup.bash
ros2 launch lab_6 video_tracking.launch.yaml pose_estimator:=0 use_ransac:=true
```

Топіки в rosbag:

/tesse/odom	ground-truth поза тіла дрона T_{body}^W (remap → /ground_truth_pose)
/tesse/left_cam/rgb/image_raw	RGB-зображення (remap → /rgb_images_topic)
/tesse/depth_cam/mono/image_raw	глибина, зареєстрована з RGB (remap → /depth_images_topic)

Примітка

У config/default.rviz дисплей зображення підписаний на /tesse/left_cam/mono/image_raw, якого в bag немає — у RViz2 переключіть його на /tesse/left_cam/rgb/image_raw. Оцінка публікується в /camera_pose, ground truth — у /gt_camera_pose.

5.2 Як влаштований вузол

Схема обробки одного кадру в cameraCallback:

1. RGB і depth синхронізуються ApproximateTime, конвертуються в cv::Mat.
2. SiftFeatureTracker::trackFeatures(prev_bgr, bgr, ...) → пари ключових точок (pts1 — попередній кадр, pts2 — поточний).
3. [D3] пікселі → bearing vectors.
4. Adapter adapter_mono(bearing_vector_1, bearing_vector_2).
5. [D4/D5] switch (pose_estimator_) — оцінка relative_pose_estimate = T_2^1 .
6. Для 2D–2D трансляція масштабується за ground truth (scaleTranslation).
7. $T_{est,t}^W = T_{gt,t-1}^W \cdot T_2^1$ (updatePoseEstimate).
8. [D4] evaluateRPE, [D4] публікація.

Ground truth камери. Симулятор дає позу тіла, а нам потрібна поза камери. У poseCallbackTesse уже зроблено

$$T_{cam}^W = T_{body}^W T_{cam}^{body}, \quad T_{cam}^{body} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 0.05 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (26)$$

де:

W	тут — світова (world) система координат симулятора, фіксована
T_{body}^W	поза тіла дрона у світі, приходить з /tesse/odom (ground truth)
T_{cam}^{body}	стала поза камери відносно тіла (extrinsics); у коді T_camera_body; 0.05 — зсув камери на 5 см по осі y тіла
T_{cam}^W	поза камери у світі — те, що публікується в /gt_camera_pose і зберігається в curr_pose_

Стовпці R_{cam}^{body} — осі камери в системі тіла: z_{cam} (оптична вісь) = x_{body} (вперед), x_{cam} (вправо на зображенні) = $-y_{body}$, y_{cam} (вниз) = $-z_{body}$. Тож curr_pose_ і prev_pose_ — уже пози **камери**, і відносна поза GT $T_2^1 = (T_{cam,t-1}^W)^{-1} T_{cam,t}^W$ є в тій самій системі, що й оцінка OpenGV.

5.3 Змінні стенсилю pose_estimation.cpp

Псевдоніми типів OpenGV (на початку файлу):

Змінна	Тип	Що означає
Adapter	CentralRelativeAdapter	обгортка над двома масивами bearing vectors (+ опційно відомі R, t); через неї всі 2D–2D алгоритми OpenGV читають дані. «Central» — одна камера (один оптичний центр) на кадр
RansacProblem	CentralRelativePoseSacProblem	«задача» для RANSAC: вміє з мінімальної вибірки порахувати $[R t]$ (NISTER / EIGHTPT / STEWENIUS / SEVENPT) і оцінити похибки точок
AdapterGivenRot	CentralRelativeAdapter	той самий адаптер; для 2-point у нього записують відоме R через setR12
RansacProblemGivenRot	TranslationOnlySacProblem	RANSAC-задача, що оцінює лише t 2-точковим розв'язувачем, беручи R з адаптера
Adapter3D	PointCloudAdapter	адаптер для двох хмар 3D-точок cloud_1, cloud_2 (відповідність за індексом)
RansacProblem3D	PointCloudSacProblem	RANSAC-задача 3D–3D; мінімальний розв'язувач — Арун на 3 точках

Поля класу PoseEstimator:

Змінна	Тип	Що означає
use_ransac_	bool	параметр ROS: true — з RANSAC, false — наївна оцінка за всіма точками (бонусні пункти)
scale_translation_	bool	параметр ROS: чи перемасштабовувати одиничну трансляцію 2D–2D до GT-довжини (для візуалізації)
show_images_	bool	параметр ROS: показувати вікна OpenCV з матчами (потрібен X11)
pose_estimator_	int	параметр ROS: вибір методу — 0 Ністер, 1 8-point, 2 2-point, 3 Арун
sf_rgb_, sf_depth_	SubscriberFilter	підписки на RGB і depth, які подаються в синхронізатор
sync_	Synchronizer	пара RGB+depth з близькими часовими мітками (ApproximateTime, черга 10) → виклик cameraCallback
feature_tracker_	unique_ptr	трекер з Lab 5 (тут SiftFeatureTracker): детекція, дескриптори, матчинг
pub_pose_estimation_	Publisher	публікує оцінену позу камери в /camera_pose

pub_pose_gt_	Publisher	публікує GT-позу камери в /gt_camera_pose
pose_sub_	Subscription	підписка на /ground_truth_pose (= /tesse/odom)
curr_pose_	PoseStamped	остання GT-поза камери у світі, $T_{cam,t}^W$; оновлюється в poseCallbackTesse
prev_pose_	PoseStamped	GT-поза камери на попередньому кадрі, $T_{cam,t-1}^W$; оновлюється в кінці cameraCallback
camera_params_	CameraParams	K — матриця K (cv::Mat 3×3), D — коефіцієнти дисторсії (тут нулі)
T_camera_body	cv::Mat 4×4	T_{cam}^{body} — стала поза камери відносно тіла
R_camera_body, t_camera_body	cv::Mat	блоки 3×3 і 3×1 матриці T_camera_body
pose_camera_body, transform_camera_body	Pose, tf2::Transform	те саме T_{cam}^{body} у форматах ROS і tf2 (для множення поз)

Локальні змінні cameraCallback:

Змінна	Тип	Що означає
rgb_msg, depth_msg	ConstSharedPtr	вхідні ROS-повідомлення поточного кадру
bgr, depth	cv::Mat	поточне кольорове зображення (8UC3) і карта глибини (32FC1, метри)
view	cv::Mat	копія bgr для малювання (не обов'язкова)
prev_bgr, prev_depth	static cv::Mat	зображення й глибина попереднього кадру; static — живуть між викликами
matched_kp_1_kp_2	std::pair	результат матчингу: .first[i] у попередньому кадрі відповідає .second[i] у поточному
N	int	кількість відповідностей (матчів)
pts1, pts2	vector<cv::Point2f>	пікселі (u, v) матчів у попередньому (1) та поточному (2) кадрі
bearing_vector_1, bearing_vector_2	bearingVectors_t	bearing vectors $f_{1,i}, f_{2,i}$ (Eigen::Vector3d, одиничні) — результат D3
adapter_mono	Adapter	адаптер OpenGV над цими двома масивами
pose_estimation	PoseStamped	оцінена абсолютна поза камери у світі, $T_{est,t}^W$, — те, що публікуємо
relative_pose_estimate	Pose	оцінена відносна поза T_2^1 — результат вашого алгоритму; за замовчуванням одинична
min_nr_of_correspondences	size_t	розмір мінімальної вибірки s методу; менше — оцінка неможлива
curr_frame, prev_frame, rotation, tf	(case 2)	GT-пози камери в tf2, їх відносне перетворення $tf = T_2^1$ і його обертання $rotation = R_2^1$, яке кладемо в адаптер
d1, d2	double (case 3)	глибина (метри) у пікселі pts1[i] попереднього і pts2[i] поточного кадру
cloud_1, cloud_2	points_t (case 3)	3D-точки $p_{1,i}, p_{2,i}$ у системах камер 1 і 2
adapter_3d	Adapter3D (case 3)	адаптер над двома хмарами
gt_t_prev_frame, gt_t_curr_frame	tf2::Transform	$T_{ref,t-1}^W$ і $T_{ref,t}^W$ — GT-пози камери

est_t_prev_frame, est_t_curr_frame	tf2::Transform	$T_{\text{est},t-1}^W$ (= GT попереднього кадру) і $T_{\text{est},t}^W$ — оцінка
---------------------------------------	----------------	---

6 Deliverable 3 — калібрування ключових точок

Deliverable 3 (5 балів)

Використати `calibrateKeypoints`, щоб перетворити пікселі на bearing vectors перед RANSAC.

Функція вже написана. Вона робить рівно те, що описано в розділі 2.1:

1. `cv::undistortPoints(pts, out, K, D)` — знімає дисторсію (тут $D = 0$) і множить на K^{-1} : на виході нормалізовані координати (x, y) на площині $Z = 1$;
2. формує $(x, y, 1)^T$ і нормує до одиничної довжини.

Треба лише викликати її у `cameraCallback`:

```
// TODO: DELIVERABLE 3 | Calibrate Keypoints
// ***** begin solution *****
calibrateKeypoints(pts1, pts2, bearing_vector_1, bearing_vector_2);
// ***** end solution *****
```

Пастка

Без цього кроку в OpenGV підуть пікселі (u, v) замість напрямків. Епіполярне обмеження 7 справедливе лише для каліброваних векторів, тож оцінки будуть безглуздими, хоча код і не впаде.

Перевірка на око: для центрального пікселя $(360, 240)$ маємо $\mathbf{f} = (0, 0, 1)$; для кута $(0, 0)$ — $\mathbf{x} = (-0.866, -0.577, 1)$, тобто горизонтальне поле зору $2 \arctan(360/415.7) \approx 82^\circ$.

7 Deliverable 4 — 2D–2D відповідності

Deliverable 4 (45 балів)

Реалізувати в `cameraCallback` 5-point (Ністер), 8-point (Longuet-Higgins) та 2-point (відоме обертання з GT) — усі з RANSAC; реалізувати `evaluateRPE`; опублікувати оцінку; побудувати 2 графіки (похибка трансляції та обертання в часі, по 3 лінії). Бонус (+5): 5-point без RANSAC.

7.1 Спільний шаблон RANSAC в OpenGV

Усі методи використовують однаковий каркас `opengv::sac::Ransac<Problem>`:

```
std::shared_ptr<Problem> problem = std::make_shared<Problem>(adapter, /* параметри */);
opengv::sac::Ransac<Problem> ransac;
ransac.sac_model_ = problem;           // std::shared_ptr (сучасний OpenGV)
ransac.threshold_ = ...;               // поріг inliers (див. розділ 2.4.2)
ransac.max_iterations_ = ...;         // верхня межа ітерацій
ransac.probability_ = 0.99;           // для адаптивної зупинки
bool ok = ransac.computeModel();
opengv::transformation_t T = ransac.model_coefficients_; // [R | t], 3x4, це T-12
std::vector<int> inliers = ransac.inliers_;
```

Поля й методи об'єкта `ransac`:

Змінна	Тип	Що означає
<code>sac_model_</code>	<code>shared_ptr</code>	задача, яку розв'язує RANSAC (мінімальний розв'язувач + функція похибки)
<code>threshold_</code>	<code>double</code>	поріг τ : точка — <code>inlier</code> , якщо її похибка менша. Для 2D–2D — у «одиницях» $1 - \cos \alpha$, для 3D–3D — у метрах
<code>max_iterations_</code>	<code>int</code>	верхня межа кількості ітерацій (за замовчуванням 1000)
<code>probability_</code>	<code>double</code>	надійність p для адаптивної зупинки: після кожної кращої гіпотези OpenGV перераховує потрібне N_{it} за формулою 15 з поточною оцінкою w
<code>iterations_</code>	<code>int</code>	скільки ітерацій фактично виконано (після <code>computeModel</code>)
<code>model_coefficients_</code>	<code>transformation_t</code>	найкраща знайдена модель — матриця 3×4 $[R t] = T_2^{-1}$
<code>inliers_</code>	<code>vector<int></code>	індекси відповідностей, що погоджуються з найкращою моделлю
<code>computeModel()</code>	<code>bool</code>	запускає RANSAC; <code>false</code> , якщо жодної моделі не знайдено

Щоб не дублювати код, додайте в клас `PoseEstimator` допоміжні методи:

```
// Поріг OpenGV для 2D-2D: сума  $(1 - \cos \alpha)$  у двох кадрах при похибці  $p_x$  пікселів
double bearingThreshold(double px) const {
    const double f = camera_params_.K.at<double>(0, 0);
    return 2.0 * (1.0 - std::cos(std::atan(px / f)));
}

template <typename Problem>
```

```

bool runRansac(const std::shared_ptr<Problem>& problem, double threshold,
               int max_iterations, opengv::transformation_t& T, bool refine = true) {
    opengv::sac::Ransac<Problem> ransac;
    ransac.sac_model_ = problem;
    ransac.threshold_ = threshold;
    ransac.max_iterations_ = max_iterations;
    ransac.probability_ = 0.99;
    if (!ransac.computeModel()) {
        RCLCPP_WARN(get_logger(), "RANSAC failed to find a model");
        return false;
    }
    // Нелінійне уточнення за всіма inliers (опціонально, але помітно покращує точність)
    if (refine) {
        problem->optimizeModelCoefficients(ransac.inliers_, ransac.model_coefficients_, T);
    } else {
        T = ransac.model_coefficients_;
    }
    RCLCPP_INFO(get_logger(), "RANSAC: %zu inliers / %zu, %d iterations",
                ransac.inliers_.size(), problem->getIndices()->size(),
                ransac.iterations_);
    return true;
}

```

де:

px	допустима похибка в пікселях (δ з розділу 2.4.2)
f	фокусна відстань у пікселях, береться з $K(0,0)$
Problem	шаблонний параметр — один з типів RansacProblem, RansacProblemGivenRot, RansacProblem3D
problem	екземпляр задачі, прив'язаний до адаптера з даними
threshold, max_iterations	значення для threshold_ i max_iterations_
T	вихід: уточнена відносна поза T_2^1 (3×4)
problem->getIndices()	індекси всіх відповідностей, з якими працює задача (тут — усі N)

Примітка

optimizeModelCoefficients мінімізує похибку перепроєкції за inliers (Левенберг–Марквардт). Якщо хочете «чистий» мінімальний розв'язувач для порівняння — замініть рядок на `T = ransac.model_coefficients_;`.

7.2 5-point (Ністер) з RANSAC — case 0

```

} else {
    // (TODO) With RANSAC
    // ***** begin solution *****
    auto problem = std::make_shared<RansacProblem>(
        adapter_mono, RansacProblem::NISTER);
    opengv::transformation_t T;
    if (runRansac(problem, bearingThreshold(1.0), 500, T)) {
        relative_pose_estimate = eigen2Pose(T);
    }
    // ***** end solution *****
}

```

Поріг $\delta = 1$ px і 500 ітерацій — з запасом відносно оцінки 146 з розділу 3.3 (OpenGV все одно зупиниться раніше завдяки адаптивному критерію з probability_). Якщо inliers замало — пробуйте $\delta = 1.5 \dots 2$ px.

7.3 5-point без RANSAC (бонус +5)

`fivept_nister(adapter)` з $N > 5$ точками бере 4-вимірне ядро **найменших квадратів** ($N \times 9$ матриця через SVD) — outliers потрапляють прямо в розв’язок. Далі до 10 кандидатів E і вибір за кількістю точок, що пройшли хіральність у `cv::recoverPose` (функція `extractPose` з `lab6_utils.h`):

```
if (!use_ransac_) {
    // ***** begin solution *****
    opengv::essential_t Es = opengv::relative_pose::fivept_nister(adapter_mono);
    cv::Mat R, t;
    // УВАГА на порядок точок: pts2, pts1 (пояснення нижче)
    extractPose(Es, pts2, pts1, camera_params_, R, t);
    relative_pose_estimate = cv2Pose(R, t);
    // ***** end solution *****
}
```

Пастка

Чому pts2, pts1? OpenGV: $f_1^T E f_2 = 0$. OpenCV `recoverPose(E, a, b, ...)` очікує $b^T E a = 0$ і повертає (R, t) з $X_b = R X_a + t$. Підставивши $a = \text{pts2}$, $b = \text{pts1}$, отримуємо $f_1^T E f_2 = 0$ ✓ і $X_1 = R X_2 + t$, тобто саме T_2^1 ✓. З порядком `pts1, pts2` ви отримаєте розклад **транспонованої** матриці — хибне обертання. Діагностика: якщо похибка обертання приблизно дорівнює подвоєному куту повороту між кадрами — у вас обернене перетворення.

7.4 8-point (Longuet-Higgins) — case 1

7.4.1 Математика

Для $N \geq 8$ відповістей розв’язуємо лінійну систему $Ae = 0$ у сенсі найменших квадратів з обмеженням масштабу:

$$\hat{e} = \arg \min_{\|e\|=1} \|Ae\|^2 = v_9, \quad (27)$$

де v_9 — правий сингулярний вектор A з найменшим сингулярним числом (або власний вектор $A^T A$ з найменшим власним числом — задача Релея). Через шум $\hat{E}$ не є essential-матрицею, тож **проектуємо** її на многовид essential-матриць (найближча за нормою Фробеніуса):

$$\hat{E} = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^T \rightarrow E = U \text{diag}(1, 1, 0) V^T. \quad (28)$$

Далі — розклад на 4 кандидати і хіральність. Відмінності від Ністера: лінійний (простий, один розв’язок), але потребує 8 точок (1177 ітерацій RANSAC проти 146 при 50% outliers), **вироджується на плоских сценах**, і обмеження $\det E = 0$ та рівність сингулярних чисел накладаються лише post hoc, а не в самому розв’язку — звідси трохи гірша точність.

7.4.2 Код

```
} else {
    // (TODO) With RANSAC
    // ***** begin solution *****
    auto problem = std::make_shared<RansacProblem>(
        adapter_mono, RansacProblem::EIGHTPT);
    opengv::transformation_t T;
    if (runRansac(problem, bearingThreshold(1.0), 2000, T)) {
        relative_pose_estimate = eigen2Pose(T);
    }
    // ***** end solution *****
}
```

Без RANSAC (опціонально):

```

if (!use_ransac_) {
    // ***** begin solution *****
    opengv::essential_t Es{opengv::relative_pose::eightpt(adapter_mono)};
    cv::Mat R, t;
    extractPose(Es, pts2, pts1, camera_params_, R, t);
    relative_pose_estimate = cv2Pose(R, t);
    // ***** end solution *****
}

```

7.5 2-point (відоме обертання) — case 2

Стенсил уже бере R_2^1 з ground truth і кладе в адаптер (`adapter_mono.setR12(rotation)`). `TranslationOnlySacProblem` на кожній ітерації викликає `relative_pose::twopt(adapter, true, indices)` — це в точності розв’язувач з Deliverable 2 (`unrotate = true` означає $\mathbf{g} = R\mathbf{f}_2$) — і повертає $[R \mid \mathbf{t}]$ з тим самим R .

```

} else {
    // (TODO) With RANSAC
    // ***** begin solution *****
    auto problem = std::make_shared<RansacProblemGivenRot>(adapter_mono);
    opengv::transformation_t T;
    // Без нелінійного уточнення: optimize_nonlinear змінив би і відоме R
    if (runRansac(problem, bearingThreshold(1.0), 200, T, false)) {
        relative_pose_estimate = eigen2Pose(T);
    }
    // ***** end solution *****
}

```

Без RANSAC (опціонально). `twopt(adapter, true)` бере лише точки 0 та 1 — випадкову пару, це погано. Краще неминимальна версія з розділу 4.2:

```

if (!use_ransac_) {
    // ***** begin solution *****
    Eigen::Matrix3d M = Eigen::Matrix3d::Zero();
    for (size_t i = 0; i < bearing_vector_1.size(); ++i) {
        const Eigen::Vector3d n = bearing_vector_1[i].cross(rotation *
bearing_vector_2[i]);
        M += n * n.transpose();
    }
    Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d> es(M);
    Eigen::Vector3d t = es.eigenvalues().col(0); // найменше власне число
    // знак — за більшістю точок з додатними глибинами
    int votes = 0;
    for (size_t i = 0; i < bearing_vector_1.size(); ++i) {
        Eigen::Matrix<double, 3, 2> A;
        A << bearing_vector_1[i], -(rotation * bearing_vector_2[i]);
        const Eigen::Vector2d lambda = A.colPivHouseholderQR().solve(t);
        votes += (lambda.x() > 0 && lambda.y() > 0) ? 1 : -1;
    }
    if (votes < 0) t = -t;
    opengv::transformation_t T;
    T.block<3, 3>(0, 0) = rotation;
    T.col(3) = t;
    relative_pose_estimate = eigen2Pose(T);
    // ***** end solution *****
}

```

де:

M	матриця $\sum_i \mathbf{n}_i \mathbf{n}_i^\top$ з розділу 4.2
n	нормаль епіполярної площини $\mathbf{n}_i = \mathbf{f}_{1,i} \times R \mathbf{f}_{2,i}$
es	розклад M на власні числа/вектори; <code>SelfAdjointEigenSolver</code> сортує власні числа за зростанням, тому стовпець 0 — найменше
t	оцінка напрямку трансляції $\hat{\mathbf{t}}$, одиничний вектор
A, lambda	матриця $(\mathbf{f}_1 - R \mathbf{f}_2)$ і глибини (λ_1, λ_2) для перевірки хіральності
votes	баланс голосів: +1 якщо точка перед обома камерами при поточному знаку t , −1 інакше

Пастка

Для 2-point передаємо `refine = false`. `TranslationOnlySacProblem::optimizeModelCoefficients` викликає `optimize_nonlinear`, який уточнює R , і $\mathbf{t}$ — відоме обертання «з'їжджає», і похибка обертання стає ненульовою (у нас було 0.08° замість 0).

7.6 Синхронізація ground truth з кадрами

У стенсиль `curr_pose_` — це **останнє** отримане повідомлення одометрії, а не поза в момент знімання кадру. На практиці це найбільше джерело похибок у цій лабі (важливіше за вибір алгоритму):

- одометрія йде з частотою 200 Гц, кадри — 20 Гц; SIFT обробляє кадр 150 мс, тож кадри чекають у черзі синхронізатора, і «остання» одометрія може бути на сотні мілісекунд новішою за кадр;
- однопотоковий еxecutor за один прохід бере по одному повідомленню з кожної підписки. Поки йде callback камери, одометрія або губиться (черга 10), або накопичує відставання на секунди (глибока черга).

Рішення (реалізовано в `pose_estimation.cpp`):

1. одометрія — в окремій callback group; `main` використовує `MultiThreadedExecutor` з 2 потоками, тож одометрія обробляється паралельно з SIFT;
2. `poseCallbackTesse` кладе кожен позу камери (з `header.stamp`) у кільцевий буфер `gt_buffer_` (під `std::mutex`), а не в `curr_pose_`;
3. на початку `cameraCallback` `lookupGtPose` бере з буфера позу, найближчу за часом до `rgb_msg->header.stamp` (крок одометрії 5 мс).

Ефект на нашому bag (медіана похибки обертання Аруна): 0.50° (остання одометрія) $\rightarrow 0.20^\circ$ (пошук за міткою, але однопотоковий еxecutor губить одометрію) $\rightarrow 0.03^\circ$ (окремий потік). Мораль: перш ніж звинувачувати алгоритм, перевірте, що еталон відповідає тому самому моменту часу.

7.7 Перший кадр

При першому виклику `prev_bgr` ініціалізується **поточним** кадром, тож обидва зображення однакові, $t = 0$ — класичне виродження. Додайте на початок `cameraCallback` (одразу після `static`-змінних):

```
current_stamp_ = rclcpp::Time(rgb_msg->header.stamp).seconds();
if (prev_pose_.header.frame_id.empty()) { // ще немає попереднього кадру з GT
    prev_bgr = bgr.clone();
    prev_depth = depth.clone();
    prev_pose_ = curr_pose_;
    return;
}
```

(`curr_pose_.header.frame_id` стає "world" після першого повідомлення одометрії, тож поки GT не прийшов, кадри теж пропускаються.)

7.8 evaluateRPE: відносна похибка пози

7.8.1 Математика

Еталонна та оцінена відносні пози:

$$T_{\text{ref},t}^{\text{ref},t-1} = (T_{\text{ref},t-1}^W)^{-1} T_{\text{ref},t}^W, \quad T_{\text{est},t}^{\text{est},t-1} = (T_{\text{est},t-1}^W)^{-1} T_{\text{est},t}^W. \quad (29)$$

У стенси́лі $T_{\text{est},t-1}^W := T_{\text{ref},t-1}^W$ і $T_{\text{est},t}^W = T_{\text{ref},t-1}^W T_2^1$, тож оцінена відносна поза — просто наш T_2^1 (з трансляцією, вже перемасштабованою `scaleTranslation`).

де:

$T_{\text{ref},t}^W$	ground-truth (reference) поза камери у світі в момент t
$T_{\text{est},t}^W$	оцінена поза камери у світі в момент t
$T_{\text{ref},t}^{\text{ref},t-1}$	справжнє переміщення камери між кадрами (GT відносна поза), у коді <code>gt_relative_pose</code>
$T_{\text{est},t}^{\text{est},t-1}$	оцінене переміщення між кадрами, у коді <code>est_relative_pose</code>
t	тут — час (номер кадру), не трансляція; трансляцію позначаємо жирним $\mathbf{t}$

Похибка трансляції. Для 2D–2D трансляція відома лише з точністю до масштабу, тому нормуємо обидва вектори:

$$\text{RPE}^{\text{tran}} = \left\| \frac{\mathbf{t}_{\text{ref}}}{\|\mathbf{t}_{\text{ref}}\|} - \frac{\mathbf{t}_{\text{est}}}{\|\mathbf{t}_{\text{est}}\|} \right\| \in [0, 2]. \quad (30)$$

Для одиничних векторів з кутом φ між ними: $\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2 \cos \varphi = 4 \sin^2(\varphi/2)$, отже

$$\text{RPE}^{\text{tran}} = 2 \sin\left(\frac{\varphi}{2}\right), \quad (31)$$

тобто $0.1 \Leftrightarrow \varphi \approx 5.7^\circ$, $0.5 \Leftrightarrow \varphi \approx 29^\circ$, $2 \Leftrightarrow$ протилежний напрям.

де:

$\mathbf{t}_{\text{ref}}, \mathbf{t}_{\text{est}}$	трансляційні частини GT та оціненої відносної пози (функція <code>trans(·)</code> із завдання)
φ	кут між напрямками $\mathbf{t}_{\text{ref}}$ і $\mathbf{t}_{\text{est}}$
$\mathbf{a}, \mathbf{b}$	довільні одиничні вектори (тут — нормовані $\mathbf{t}_{\text{ref}}, \mathbf{t}_{\text{est}}$)

Похибка обертання — хордальна відстань:

$$\text{RPE}^{\text{rot}} = \|R_{\text{ref}} - R_{\text{est}}\|_F. \quad (32)$$

Зв'язок з кутом: нехай θ — кут обертання $R_{\text{ref}}^\top R_{\text{est}}$. Тоді

$$\begin{aligned} \|R_{\text{ref}} - R_{\text{est}}\|_F^2 &= \text{tr}\left((R_{\text{ref}} - R_{\text{est}})^\top (R_{\text{ref}} - R_{\text{est}})\right) = 6 - 2 \text{tr}(R_{\text{ref}}^\top R_{\text{est}}) \\ &= 6 - 2(1 + 2 \cos \theta) = 8 \sin^2\left(\frac{\theta}{2}\right), \end{aligned} \quad (33)$$

(використано $\text{tr } R = 1 + 2 \cos \theta$ для обертання на кут θ), отже

$$\text{RPE}^{\text{rot}} = 2\sqrt{2} \sin\left(\frac{\theta}{2}\right), \quad \theta = 2 \arcsin\left(\frac{\text{RPE}^{\text{rot}}}{2\sqrt{2}}\right). \quad (34)$$

Орієнтири: $1^\circ \Leftrightarrow 0.0247$, $0.1^\circ \Leftrightarrow 0.00247$. На графіках зручніше показувати θ у градусах — критерії в завданні сформульовані саме в градусах.

де:

$R_{\text{ref}}, R_{\text{est}}$	обертальні частини GT та оціненої відносної пози (функція <code>rot(·)</code> із завдання)
$\ \cdot\ _F$	норма Фробеніуса: $\ A\ _F = \sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\text{tr}(A^\top A)}$
θ	кут «залишкового» обертання $R_{\text{ref}}^\top R_{\text{est}}$ — наскільки оцінене обертання відрізняється від справжнього

7.8.2 Код

Додайте поля класу та ініціалізацію в конструкторі (потрібні `#include <filesystem>`, `#include <array>` і `#include <iomanip>`):

```
std::ofstream rpe_log_;
double current_stamp_ = 0.0;
```

```
// у конструкторі, після зчитування pose_estimator_ і use_ransac_
declare_parameter<std::string>("rpe_log_dir", "/workspace/VNAV-labs/lab6/results");
const std::string log_dir = get_parameter("rpe_log_dir").as_string();
std::filesystem::create_directories(log_dir);
static const std::array<const char*, 4> kNames{"5pt", "8pt", "2pt", "arun"};
if (pose_estimator_ >= 0 && pose_estimator_ < 4) {
    const std::string file = log_dir + "/rpe_" + kNames[pose_estimator_] +
                             (use_ransac_ ? "" : "_noransac") + ".csv";
    rpe_log_.open(file);
    rpe_log_ << "stamp,trans_err,rot_err_chordal,rot_err_deg,gt_trans_norm\n";
    RCLCPP_INFO(get_logger(), "Logging RPE to %s", file.c_str());
}
```

де:

rpe_log_	файловий потік CSV з похибками, по рядку на кадр
current_stamp_	часова мітка поточного RGB-кадру в секундах (для осі часу на графіках)
log_dir	каталог для CSV (параметр rpe_log_dir)
kNames	короткі назви методів для імені файлу, індекс = pose_estimator_

Тіло evaluateRPE:

```
// ***** begin solution *****
const Eigen::Isometry3d est = tf2TransformToIsometry(est_relative_pose);
const Eigen::Isometry3d gt = tf2TransformToIsometry(gt_relative_pose);

// Обертання: хордальна відстань (Eigen::Matrix::norm() – це норма Фробеніуса)
const double rot_err = (gt.rotation() - est.rotation()).norm();
const double rot_err_deg =
    2.0 * std::asin(std::min(1.0, rot_err / (2.0 * std::sqrt(2.0)))) * 180.0 / M_PI;

// Трансляція: для 2D-2D порівнюємо лише напрямки (одичні вектори)
Eigen::Vector3d t_gt = gt.translation();
Eigen::Vector3d t_est = est.translation();
const double gt_norm = t_gt.norm();
const bool normalize = pose_estimator_ < 3;
if (normalize) {
    if (gt_norm < 1e-6 || t_est.norm() < 1e-6) {
        RCLCPP_WARN(get_logger(), "Translation too small to normalize, skipping frame");
        return;
    }
    t_gt.normalize();
    t_est.normalize();
}
const double trans_err = (t_gt - t_est).norm();

RCLCPP_INFO(get_logger(), "RPE: trans %.4f | rot %.5f (%.3f deg)",
    trans_err, rot_err, rot_err_deg);
if (rpe_log_.is_open()) {
    rpe_log_ << std::fixed << std::setprecision(6) << current_stamp_ << ", "
        << trans_err << ", " << rot_err << ", " << rot_err_deg << ", "
        << gt_norm << "\n";
}
```

```

    rpe_log_.flush();
}
// ***** end solution *****

```

де:

<code>est, gt</code>	<code>Eigen::Isometry3d</code> — оцінена та GT відносні пози (4×4 жорстке перетворення)
<code>rot_err</code>	хордальна похибка обертання $\ R_{\text{ref}} - R_{\text{est}}\ _F$
<code>rot_err_deg</code>	та сама похибка, переведена в кут θ у градусах
<code>t_gt, t_est</code>	трансляції $t_{\text{ref}}, t_{\text{est}}$ (після нормування — одиничні)
<code>gt_norm</code>	довжина справжнього зсуву між кадрами в метрах (до нормування)
<code>normalize</code>	чи порівнювати лише напрями (для 2D–2D методів — так)
<code>trans_err</code>	похибка трансляції RPE^{tran}

Примітка

Колонка `gt_trans_norm` (довжина GT-трансляції між кадрами) корисна для аналізу: великі похибки напрямку майже завжди припадають на кадри, де дрон ледь зрушив — це виродження $t \approx 0$ з розділу 3.2, а не баг.

7.9 Публікація оцінки

```

// ***** begin solution *****
pose_estimation.header.frame_id = "world";
pose_estimation.header.stamp = rgb_msg->header.stamp;
pub_pose_estimation->publish(pose_estimation);
// ***** end solution *****

```

`pose_estimation.pose` уже обчислена `updatePoseEstimate` як $T_{\text{ref}, t-1}^W T_2^1$. У RViz2 (fixed frame world) стрілка `/camera_pose` має майже збігатися з `/gt_camera_pose`.

Пастка

Ця візуалізація «шахраює»: використовує GT попереднього кадру і GT-масштаб трансляції. Ланцюжок лише з 2D–2D оцінок побудувати не можна — кожна має свій невідомий масштаб.

8 Deliverable 5 — 3D–3D відповідності (алгоритм Аруна)

Deliverable 5 (20 балів)

За глибиною перетворити ключові точки на 3D-хмари, оцінити відносну позу алгоритмом Аруна з RANSAC, побудувати графіки похибок.

8.1 Від пікселя до 3D-точки

Стенсил уже це робить: $\mathbf{f} \rightarrow \mathbf{f}/f_z$ (нормалізовані координати, $z = 1$), потім множення на глибину d :

$$\mathbf{p} = d \cdot (x, y, 1)^\top = dK^{-1}(u, v, 1)^\top. \quad (35)$$

Тут d — **глибина по осі Z**, а не відстань вздовж променя; тому ділимо саме на f_z , а не просто множимо одиничний вектор на d .

де:

f_z третя (z -) компонента bearing vector $\mathbf{f}$ (у коді `bearing_vector_1[i](2, 0)`); не плутати з фокусною відстанню
 d значення карти глибини в пікселі ключової точки (`d1, d2` у коді), метри
 $(x, y, 1)$ нормалізовані координати точки

8.2 Математика алгоритму Аруна

Дано пари $(\mathbf{p}_{1,i}, \mathbf{p}_{2,i})$, $i = 1..N$. Шукаємо

$$(\mathbf{R}^*, \mathbf{t}^*) = \arg \min_{\mathbf{R} \in \text{SO}(3), \mathbf{t} \in \mathbb{R}^3} \sum_{i=1}^N \|\mathbf{p}_{1,i} - \mathbf{R}\mathbf{p}_{2,i} - \mathbf{t}\|^2. \quad (36)$$

де:

$\mathbf{p}_{1,i}, \mathbf{p}_{2,i}$ 3D-координати i -ї точки в камері 1 (`cloud_1[i]`) і камері 2 (`cloud_2[i]`)
 $\mathbf{R}^*, \mathbf{t}^*$ шукані оптимальні обертання й трансляція (T_2^1); зірочка — «оптимальне значення»
 $\text{SO}(3)$ група обертань: матриці 3×3 з $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = +1$
 N кількість пар точок (у RANSAC — 3 для мінімальної вибірки або всі inliers)

Крок 1: трансляція. Похідна за $\mathbf{t}$: $-2 \sum_i (\mathbf{p}_{1,i} - \mathbf{R}\mathbf{p}_{2,i} - \mathbf{t}) = 0$, тому

$$\mathbf{t}^* = |\mathbf{p}|_1 - \mathbf{R}|\mathbf{p}|_2, \quad |\mathbf{p}|_k = \frac{1}{N} \sum_i \mathbf{p}_{k,i}. \quad (37)$$

Тобто оптимальна трансляція суміщає центроїди. Тут $|\mathbf{p}|_1, |\mathbf{p}|_2$ — центроїди (середні точки) хмар 1 і 2.

Крок 2: обертання. Підставимо $\mathbf{t}^*$ і позначимо центровані точки $\mathbf{a}_i = \mathbf{p}_{1,i} - |\mathbf{p}|_1$, $\mathbf{b}_i = \mathbf{p}_{2,i} - |\mathbf{p}|_2$:

$$\sum_i \|\mathbf{a}_i - \mathbf{R}\mathbf{b}_i\|^2 = \sum_i (\|\mathbf{a}_i\|^2 + \|\mathbf{b}_i\|^2) - 2 \sum_i \mathbf{a}_i^\top \mathbf{R}\mathbf{b}_i. \quad (38)$$

(Використано $\|\mathbf{R}\mathbf{b}\| = \|\mathbf{b}\|$.) Перша сума від $\mathbf{R}$ не залежить, тож треба **максимізувати**

$$\sum_i \mathbf{a}_i^\top \mathbf{R}\mathbf{b}_i = \text{tr} \left(\mathbf{R} \sum_i \mathbf{b}_i \mathbf{a}_i^\top \right) = \text{tr}(\mathbf{R}\mathbf{H}), \quad \mathbf{H} = \sum_i \mathbf{b}_i \mathbf{a}_i^\top \in \mathbb{R}^{3 \times 3}. \quad (39)$$

де:

$\mathbf{a}_i, \mathbf{b}_i$ точки хмар 1 і 2, зсунуті так, що їхні центроїди в нулі
 $\mathbf{H}$ матриця крос-коваріації 3×3 між центрованими хмарами; вся інформація про обертання — в ній

Крок 3: SVD. Нехай $\mathbf{H} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$, $\mathbf{S} = \text{diag}(s_1, s_2, s_3)$, $s_k \geq 0$. Тоді

$$\text{tr}(RH) = \text{tr}(RUSV^\top) = \text{tr}\left(\underbrace{V^\top RU}_=:Q S\right) = \sum_k Q_{kk} s_k \leq \sum_k s_k, \quad (40)$$

бо Q ортогональна і $|Q_{kk}| \leq 1$. Тут U, V — ортогональні матриці SVD матриці H (не ті, що в розкладі E), S — діагональна матриця сингулярних чисел $s_1 \geq s_2 \geq s_3 \geq 0$ (не плутати з розміром вибірки s), $Q = V^\top RU$ — допоміжна ортогональна матриця. Рівність досягається при $Q = I$, тобто

$$R^* = VU^\top. \quad (41)$$

Крок 4: відбиття. Якщо $\det(VU^\top) = -1$, це відбиття, а не обертання (трапляється при шумних або майже компланарних даних). Тоді найкраще **обертання** —

$$R^* = V \text{diag}(1, 1, \det(VU^\top)) U^\top. \quad (42)$$

Мінімальна вибірка — 3 неколінеарні точки (6 ступенів свободи, кожна точка дає 3 рівняння; 2 точки не фіксують обертання навколо прямої, що їх з'єднує). **Виродження:** колінеарні точки. На відміну від 2D–2D, масштаб визначений повністю — трансляція в метрах.

8.3 Код — case 3

```

} else {
    // (TODO) With RANSAC
    // ***** begin solution *****
    auto problem = std::make_shared<RansacProblem3D>(adapter_3d);
    opengv::transformation_t T;
    // Попіг – евклідова відстань у метрах між p1 та R*p2 + t
    if (runRansac(problem, 0.05, 300, T)) {
        relative_pose_estimate = eigen2Pose(T);
    }
    // ***** end solution *****
}

```

Без RANSAC (опціонально):

```

if (!use_ransac_) {
    // ***** begin solution *****
    const opengv::transformation_t T =
    opengv::point_cloud::threept_arun(adapter_3d);
    relative_pose_estimate = eigen2Pose(T);
    // ***** end solution *****
}

```

`PointCloudAdapter(cloud_1, cloud_2)` шукає T з $p_1 = Rp_2 + t$, тобто знову T_2^1 — конвенція та сама, що й для 2D–2D. Для Аруна `scaleTranslation` не викликається (умова `pose_estimator_ < 3`), бо масштаб справжній.

8.4 Невалідна глибина

Пікселі без глибини (NaN, $+\infty$, 0) дають точки в нулі або на нескінченності. RANSAC їх здебільшого відкине, але краще не пускати в адаптер взагалі. Замініть побудову хмар у стенсилі:

```

opengv::points_t cloud_1, cloud_2;
for (int i = 0; i < N; i++) {
    const double d1 = prev_depth.at<float>(std::floor(pts1[i].y),
std::floor(pts1[i].x));
    const double d2 = depth.at<float>(std::floor(pts2[i].y), std::floor(pts2[i].x));
    if (!std::isfinite(d1) || !std::isfinite(d2) || d1 <= 0.0 || d2 <= 0.0)
        continue;
}

```

```
cloud_1.push_back(bearing_vector_1[i] / bearing_vector_1[i].z() * d1);  
cloud_2.push_back(bearing_vector_2[i] / bearing_vector_2[i].z() * d2);  
}
```

Порада

Перевірте кодування глибини: `ros2 topic echo --once /tesse/depth_cam/mono/image_raw --field encoding`. Код розраховує на 32FC1 (метри). Якщо там 16UC1 (міліметри) — читайте `at<uint16_t>` і діліть на 1000.

Порівняння трансляцій. Для Аруна `evaluateRPE` (з кодом вище) рахує похибку трансляції **в метрах** без нормування. Оскільки кадри дуже часті, зсув між ними — сантиметри, і похибка буде дуже малою. Якщо хочете порівняти з 2D–2D методами на одній шкалі — додатково запишіть нормовану версію (змінивши умову `normalize`), але пам’ятайте: при крихітній базі напрям шумний.

9 Запуск, графіки, перевірка результатів

9.1 Запуск усіх методів

Кожен запуск пише свій CSV у VNAV-labs/lab6/results/. Дочекайтеся кінця bag (процес `ros2 bag play` завершиться), потім `Ctrl+C`:

```
ros2 launch lab_6 video_tracking.launch.yaml pose_estimator:=0 # → rpe_5pt.csv
ros2 launch lab_6 video_tracking.launch.yaml pose_estimator:=1 # → rpe_8pt.csv
ros2 launch lab_6 video_tracking.launch.yaml pose_estimator:=2 # → rpe_2pt.csv
ros2 launch lab_6 video_tracking.launch.yaml pose_estimator:=3 # → rpe_arun.csv
# бонус:
ros2 launch lab_6 video_tracking.launch.yaml pose_estimator:=0 use_ransac:=false
```

Примітка

SIFT на кожному кадрі повільний; якщо вузол не встигає за bag, частина кадрів пропускається (синхронізатор має чергу 10). Для графіків це не критично. Можна сповільнити програвання, змінивши у launch-файлі команду на `ros2 bag play ... --clock --rate 0.5`.

9.2 Скрипт для графіків

Збережіть як VNAV-labs/lab6/scripts/plot_rpe.py (його також треба здати):

```
#!/usr/bin/env python3
"""Графіки RPE для Lab 6: python3 plot_rpe.py [results_dir]"""
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

RESULTS = Path(sys.argv[1] if len(sys.argv) > 1 else "VNAV-labs/lab6/results")
METHODS_2D = {"5pt": "5-point (Nistér)", "8pt": "8-point (Longuet-Higgins)",
              "2pt": "2-point (known R)", "5pt_noransac": "5-point, no RANSAC"}

def load(name):
    path = RESULTS / f"rpe_{name}.csv"
    if not path.exists():
        return None
    df = pd.read_csv(path)
    df["time"] = df["stamp"] - df["stamp"].iloc[0]
    return df

def plot(methods, column, ylabel, title, out, logy=False):
    fig, ax = plt.subplots(figsize=(10, 4))
    for key, label in methods.items():
        df = load(key)
        if df is not None:
            ax.plot(df["time"], df[column], lw=0.9, label=label)
    ax.set(xlabel="час, c", ylabel=ylabel, title=title)
    if logy:
        ax.set_yscale("log")
    ax.grid(alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(RESULTS / out, dpi=150)
```

```
def summary(methods):
    for key, label in methods.items():
        df = load(key)
        if df is None:
            continue
        tr, rot = df.trans_err, df.rot_err_deg
        print(f"{label:28s} n={len(df):4d}")
        print(f"  trans: median={tr.median():.4f} "
              f"<0.5: {(tr < 0.5).mean():.1%} <0.1: {(tr < 0.1).mean():.1%}")
        print(f"  rot: median={rot.median():.3f} deg "
              f"<1: {(rot < 1).mean():.1%} <0.1: {(rot < 0.1).mean():.1%}")

plot(METHODS_2D, "trans_err", "RPE трансляції (одичні вектори)",
     "2D-2D: похибка напрямку трансляції", "rpe_trans_2d.png")
plot(METHODS_2D, "rot_err_deg", "RPE обертання, °",
     "2D-2D: похибка обертання", "rpe_rot_2d.png", logy=True)
plot({"arun": "Arun 3-point"}, "trans_err", "RPE трансляції, м",
     "3D-3D: похибка трансляції", "rpe_trans_3d.png")
plot({"arun": "Arun 3-point"}, "rot_err_deg", "RPE обертання, °",
     "3D-3D: похибка обертання", "rpe_rot_3d.png")
summary(**METHODS_2D, "arun": "Arun 3-point")
```

де:

RESULTS

METHODS_2D

df

stamp, trans_err, rot_err_chordal, rot_err_deg, gt_trans_norm

tr, rot

каталог з CSV (аргумент командного рядка)
словник «суфікс файлу → підпис у легенді»
таблиця pandas з одного CSV;
додана колонка **time** — секунди від першого кадру
колонки CSV: час кадру, RPE^{tran} , RPE^{rot} , кут θ у градусах, довжина GT-зсуву (м)
скорочення для колонок **trans_err** і **rot_err_deg** у підсумковій статистиці

Примітка

У 2-point обертання взято з GT, тож його похибка обертання ≈ 0 (лише шум синхронізації). Це очікувано; на логарифмічній шкалі лінія буде біля нуля. Логарифмічна шкала взагалі зручна для обертання — значення відрізняються на порядки.

9.3 Чого очікувати

Метод	Обертання	Трансляція
5-point + RANSAC	$< 1^\circ$ для більшості кадрів	< 0.5 ; трохи краще за 8-point
8-point + RANSAC	$< 1^\circ$ для більшості кадрів	< 0.5
2-point + RANSAC	≈ 0 (з GT)	< 0.1 для більшості кадрів
Арун + RANSAC	$< 0.1^\circ$	< 0.1 (нормовано), ще менше в метрах

5-point без RANSAC	великі викиди	часто близько до 1–2 (хаотичний напрям)
--------------------	---------------	---

Як пояснити результати у звіті:

- **5pt vs 8pt:** 5-point накладає всі обмеження essential-матриці точно, а мінімальна вибірка менша — більше шансів на чисту вибірку за той самий бюджет ітерацій; 8-point проектує на багатовид post hoc і чутливіший до шуму та плоских ділянок сцени (стіни офісу!).
- **2pt найкращий по трансляції,** бо з 5 невідомих лишилось 2 — вся інформація з точок іде на напрям t ; плюс мінімальна вибірка з 2 точок.
- **Арун найточніший:** глибина додає метричну інформацію, задача лінійна з замкненим розв'язком, немає масштабної невизначеності.
- **Сплески похибки трансляції** припадають на кадри з малим `gt_trans_norm` (дрон майже нерухомий) — виродження $t \approx 0$.
- **Без RANSAC** — наочна демонстрація, що least squares не витримує outliers SIFT.

9.4 Фактичні результати на `vnav-lab6-office`

Усі методи з RANSAC, поріг 1 px (2D–2D) та 5 см (3D–3D), нелінійне уточнення увімкнене (крім 2-point). Прогін у Docker на Apple Silicon (емуляція x86): вузол встигає обробити 5–8 кадрів/с з 20, тож «сусідні» кадри в CSV віддалені в середньому на 7 см. Статистику дає `plot_rpe.py`.

Метод	кадрів	трансл., медіа-на	трансл. < 0.5 / < 0.1	оберт., медіана	оберт. < 1° / < 0.1°
5-point	1800	0.277	65% / 22%	0.33°	92% / 15%
8-point	1561	0.184	84% / 28%	0.20°	98% / 21%
2-point	1844	0.062	97% / 70%	0°	100% / 100%
Арун	961	0.003 м	100% / 100%	0.03°	100% / 97%

Висновки.

- Обертання для 5pt/8pt — у межах очікувань курсу ($< 1^\circ$ для більшості кадрів); Арун — $< 0.1^\circ$ для 97% кадрів; 2-point — трансляція < 0.1 для 70% кадрів.
- Похибка напряму трансляції 2D–2D майже повністю визначається довжиною бази: для 5-point медіана 0.79 при кроці < 4 см і 0.11 при кроці > 9 см (8-point: 0.44 і 0.09). Це пряма ілюстрація виродження $t \rightarrow 0$ з розділу 3.2.
- На цьому прогоні 8-point дещо точніший за 5-point в усіх діапазонах бази — всупереч очікуванню курсу. Ймовірні причини: дуже мала база (5-point з мінімальної вибірки частіше обирає хибний з кількох розв'язків, а 8-point усереднює більше точок ще на етапі гіпотези), і однаковий для обох поріг 1 px. Це варто чесно зазначити у звіті.



Рисунок 2: Похибка напрямку трансляції, 2D–2D.

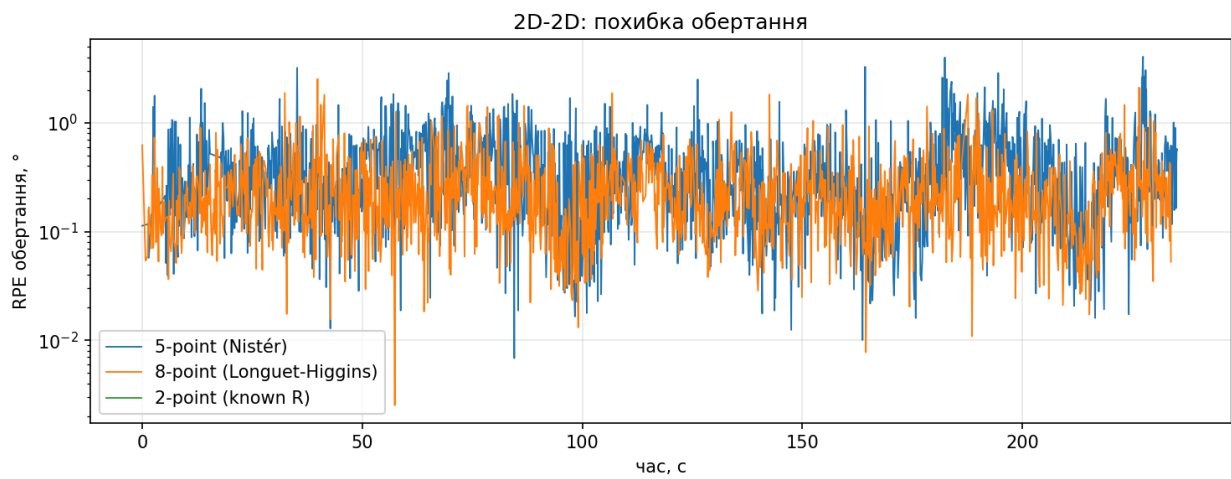


Рисунок 3: Похибка обертання, 2D–2D (лог. шкала; 2-point = 0 і не відображається).

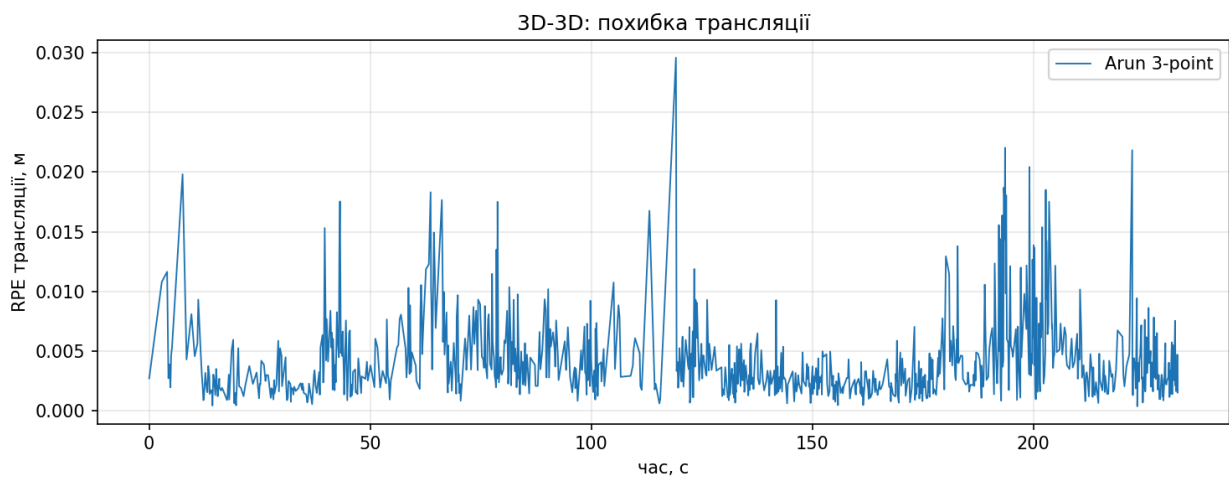


Рисунок 4: Арун: похибка трансляції, метри.

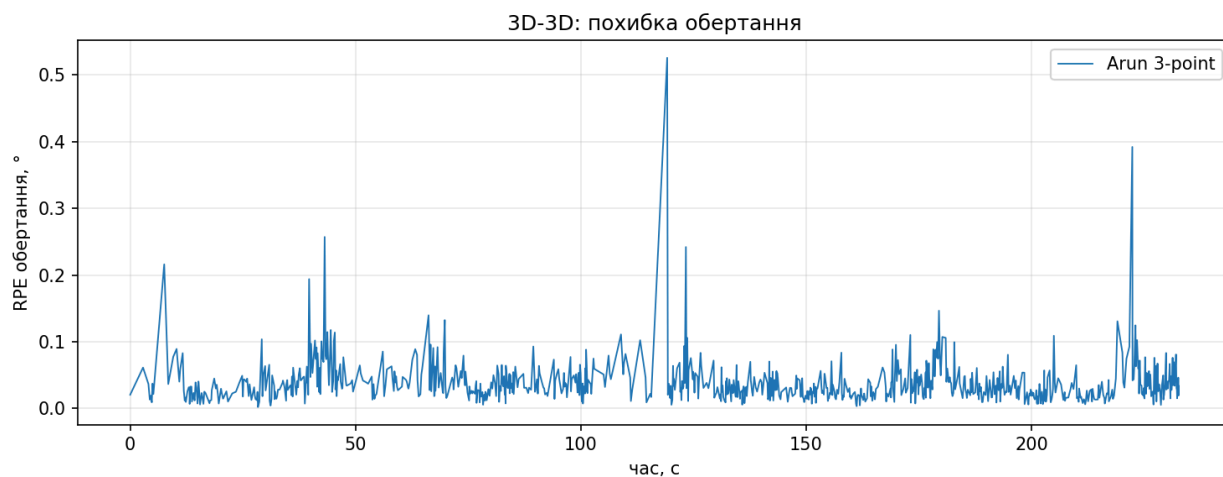


Рисунок 5: Арун: похибка обертання.

9.5 Налаштування: типові симптоми

Симптом	Ймовірна причина
Похибка трансляції ≈ 2 (напрям протилежний)	Переплутано кадри 1/2 або використано T_1^2 замість T_2^1
Похибка обертання $\approx$ подвоєний кут між кадрами	Обернене перетворення (порядок <code>pts2</code> , <code>pts1</code> в <code>extractPose</code>)
Усі похибки великі, <code>inliers</code> мало	Забули <code>calibrateKeypoints</code> (пікселі замість <code>bearing vectors</code>) або поріг надто малий
Великий сплеск на першому кадрі	Не пропущено перший кадр (однакові зображення)
Арун: трансляція величезна	Невалідна глибина (<code>inf/NaN</code>) або неправильне кодування глибини
<code>/camera_pose</code> не видно в RViz2	Не заповнено <code>header.frame_id = "world"</code> або не викликано <code>publish</code>

9.6 Чекліст здачі

- **Індивідуально (Gradescope, LaTeX/Typst):** D1 — кроки Ністера, виродження, $\mathbb{E}[k] = 32$ та $N = 146$; D2 — виведення 2-point розв'язувача, формула 25, вибір знаку, виродження; (бонус) псевдокод RANSAC.
- **Команда (репозиторій, папка lab6):** повний пакет `lab6` з кодом; PDF з 4 графіками (2D–2D: трансляція і обертання по 3 лінії; Арун: трансляція і обертання) та коментарями; `plot_rpe.py`; (бонус) лінія 5-point без RANSAC.

А Словник позначень

Зведена таблиця всіх математичних символів гайду. Деякі літери в літературі традиційно мають кілька значень — такі випадки позначені (!), значення визначається контекстом.

А.1 Системи координат і пози

Символ	Значення	Розділ
W (!)	світова система координат; також матриця повороту на 90° у розкладі E і базисна матриця в алгоритмі Ністера	2.3, 3.1, 5.2
T_B^A	поза системи B у системі A , елемент $SE(3)$ (матриця 4×4 або $3 \times 4 [R \mid t]$)	1.1
R_B^A, t_B^A	обертальна і трансляційна частини T_B^A	1.1
$R = R_2^1$	відносне обертання камери 2 (поточний кадр) відносно камери 1 (попередній)	2.2
$t = t_2^1$	положення центру камери 2 в системі камери 1 — базова лінія	2.2
t (!)	час / номер кадру (звичайний шрифт); трансляція — жирне t	7.7
$T_{\text{body}}^W, T_{\text{cam}}^{\text{body}}, T_{\text{cam}}^W$	поза тіла у світі, камери відносно тіла, камери у світі	5.2
$T_{\text{ref},t}^W, T_{\text{est},t}^W$	GT і оцінена поза камери у світі в момент t	7.7
$SE(3), SO(3)$	група жорстких перетворень / група обертань у 3D	1.1, 8.2
O_1, O_2	оптичні центри камери в кадрах 1 і 2	2.2

А.2 Камера та вимірювання

Символ	Значення	Розділ
P	3D-точка сцени	2.2
$p = (X, Y, Z)^\top$	координати точки в системі камери; Z — глибина	2.1
p_1, p_2	координати тієї ж точки в камері 1 і 2	2.2
(u, v)	піксельні координати (стовпець, рядок)	2.1
K	матриця внутрішніх параметрів камери	2.1
$f_x, f_y; f$ (!)	фокусні відстані в пікселях; $f = f_x$ у формулі порогу	2.1, 2.4
c_x, c_y	головна точка	2.1
$\lambda, \lambda_1, \lambda_2$	проективний множник / глибини вздовж променів (для хіральності)	2.1, 2.3
$x = (x, y, 1)^\top$	нормалізовані координати на площині $Z = 1$	2.1
f, f_1, f_2	bearing vector(и) — одиничні напрямки на точку	2.1
f_z	z -компонента bearing vector	8.1
d	глибина з depth-зображення, метри	8.1
q_i	калібровані координати точки (позначення Ністера)	3.1

А.3 Епіполярна геометрія

Символ	Значення	Розділ
E	essential-матриця, $f_1^\top E f_2 = 0$	2.2
$[a]_\times$	кососиметрична матриця векторного добутку	1.1
σ	ненульове сингулярне число E	2.2
e	елементи E одним вектором $\mathbb{R}^9$	2.2

a_i, A	рядок коефіцієнтів i -ї відповідності і матриця системи $Ae = 0$	2.2
$\otimes$	добуток Кронекера	2.2
U, V (!)	ортогональні матриці SVD (для E у 2.3, для H у 8.2)	2.3, 8.2
u_3	третій стовпець U — напрям t	2.3
v_9	правий сингулярний вектор A з найменшим сингулярним числом	7.4
$\hat{E}$	оцінка E методом найменших квадратів (до проєкції на многовид)	7.4

A.4 RANSAC

Символ	Значення	Розділ
ε, w	частка outliers / inliers ($w = 1 - \varepsilon$)	2.4
s (!)	розмір мінімальної вибірки; не плутати з сингулярними числами s_k в алгоритмі Аруна	2.4
$q = w^s$	імовірність чистої вибірки	2.4
$k, \mathbb{E}[k]$	номер першої чистої вибірки та його очікуване значення	2.4
p	бажана надійність RANSAC	2.4
N_{it}	потрібна кількість ітерацій	2.4
N	кількість відповідей	2.2
τ	поріг inlier	2.4
$\alpha_k, \hat{f}_k$	кутова похибка і перепроєктований напрям у кадрі k	2.4
δ	допустима похибка в пікселях	2.4

A.5 Алгоритм Ністера

Символ	Значення	Розділ
X, Y, Z, W	базисні матриці ядра A (3×3)	3.1
x, y, z (!)	невідомі коефіцієнти лінійної комбінації ($w = 1$); не координати точки	3.1
M (!)	матриця 10×20 коефіцієнтів кубічних рівнянь; у розділі 4.2 — інша M (сума $n_i n_i^\top$)	3.1, 4.2
m	вектор 20 мономів	3.1
$B(z)$	матриця 3×3 поліномів від z ; $\det B(z)$ — поліном 10-го степеня	3.1
z_k, E_k	k -й дійсний корінь та відповідна essential-матриця	3.1

A.6 2-точковий розв’язувач

Символ	Значення	Розділ
$g_i = Rf_{2,i}$	промінь кадру 2, розвернутий в осі камери 1	4.2
n_i	нормаль епіполярної площини i -ї точки	4.2
$\hat{t}$	одиничний напрям трансляції	4.2
β	кут між t та епіполярною площиною (міра похибки)	4.3

A.7 Алгоритм Аруна

Символ	Значення	Розділ
R^*, t^*	оптимальні обертання і трансляція	8.2
$ p _1, p _2$	центроїди хмар	8.2

$\mathbf{a}_i, \mathbf{b}_i$	центровані точки хмар 1 і 2	8.2
H	матриця крос-коваріації $\sum_i \mathbf{b}_i \mathbf{a}_i^\top$	8.2
S, s_k	діагональ сингулярних чисел H	8.2
$Q = V^\top R U$	допоміжна ортогональна матриця в доведенні	8.2

A.8 Метрики

Символ	Значення	Розділ
RPE^{tran}	похибка напрямку (2D–2D) або вектора (3D–3D) трансляції	7.7
RPE^{rot}	хордальна похибка обертання $\ R_{\text{ref}} - R_{\text{est}}\ _F$	7.7
$\mathbf{t}_{\text{ref}}, \mathbf{t}_{\text{est}}, R_{\text{ref}}, R_{\text{est}}$	частини GT та оціненої відносних поз	7.7
φ	кут між напрямками трансляцій	7.7
θ	кут залишкового обертання $R_{\text{ref}}^\top R_{\text{est}}$	7.7
$\ \cdot\ _F, \text{tr}$	норма Фробеніуса, слід матриці	7.7, 2.2

Джерела: завдання курсу VNAV (MIT, CC BY 4.0); D. Nistér, CVPR 2003; K. S. Arun, T. S. Huang, S. D. Blostein, *Least-squares fitting of two 3-D point sets*, IEEE TPAMI 1987; L. Kneip, OpenGV.